The FAIR Agentic Framework

A Comprehensive Guide

Flexible · Agnostic · Interoperable · Reasoning

Code-aligned revision — `fairlib` and `fair_llm_demos` snapshots
(April 2026)

Contents

0.1 Introduction: From Language Models to Autonomous Agents 1
0.2 What You Are Really Learning Here 1
0.3 How to Use This Guide 2
0.4 A Roadmap Toward Understanding 3

1 The FAIR Agentic Framework Explained 5
1.1 What Is an AI Agent? (And Why You Need a Framework) 5
1.2 Introducing the FAIR Framework 6
1.2.1 Why FAIR Is Friendly to New Builders 7
1.3 The Anatomy of a FAIR Agent 7
1.4 Understanding MCP and Why It Matters 8
1.4.1 What MCP Is 8
1.4.2 The Basic MCP Mental Model 9
1.4.3 How MCP Fits into FAIR 9
1.5 How It Works: The ReAct Cycle in Action 10
1.6 Getting Started: Your First FAIR Agent 10
1.6.1 A Minimal First Example 11
1.6.2 How to Learn Faster from the Demos 11
1.7 What Comes Next 12

2 Under the Hood - How a FAIR Agent Thinks and Acts 13
2.1 The Core Loop: A Walk Through SimpleAgent.arun() 13
2.2 The Planner: The Agent's "*Brain*" 14
2.3 The Art of Prompt Engineering: PromptBuilder and Prompt Items . 15
2.3.1 Anatomy of a Prompt 15
2.3.2 The "Magic" Revealed: Deconstructing a Default Prompt . . . 16
2.4 Developer Customization: How to Change Your Agent's "Personality" 17

2.4.1 Changing the Agent's Role (The Easiest & Most Common Override) . . . 17
2.4.2 Teaching the Agent New Tricks (Changing the Examples) . . . 18
2.5 From Thought to Action: The ToolExecutor . . . 19
2.5.1 Completing the Loop: Memory and Observation . . . 19

3 Advanced Capabilities - From Single Agents to Collaborative Systems 20
3.1 Multi-Agent Collaboration: The "Committee of Agents" . . . 20
3.1.1 Philosophy: . . . 20
3.1.2 Technical Implementation: . . . 21
3.2 Grounding Agents in Reality: Advanced RAG . . . 23
3.2.1 Philosophy: . . . 23
3.2.2 The Technical RAG Pipeline: . . . 23
3.3 Enforcing Structure: Reliable JSON Output and Pydantic Validation 24
3.3.1 Philosophy: . . . 24
3.3.2 Technical Implementation (demo_structured_output.py): . . . 24
3.4 Practical Security Considerations . . . 26
3.4.1 Philosophy: . . . 26
3.4.2 Technical Implementation: . . . 26

4 The Expert's Guide - Building a Self-Diagnosing Agent 27
4.1 The Architectural Blueprint - A Committee of Diagnosticians . . . 27
4.2 Engineering the Tools for Diagnosis . . . 28
4.2.1 The CodeExecutionTool . . . 28
4.2.2 The FileContentTool (Hypothetical) . . . 28
4.3 The Master Plan: Orchestration with the DiagnosisManager . . . 29
4.4 Assembling the Committee in Code . . . 31
4.5 . . . 32
4.6 Conclusion: A Framework for Complex Problem-Solving . . . 32

5 Core Framework Modules (A Deep Dive) 34
5.1 The Architectural Heart: core/interfaces/ . . . 34
5.2 The Data of Conversation: core/message.py and core/types.py . . . 35
5.3 The Brain: The Cognitive and Planning Modules . . . 36

5.4 The Agent's Body: Perception, Action, and Memory 37
5.5 The Gateway to the World: The Model Abstraction Layer (modules/mal/) 38
5.6 The MCP Integration Layer (modules/mcp/) 38
5.7 The Developer's Toolkit: the `fairlib` Import Surface and the Demos 39

6 The Prompt Engineering Workbench - Advanced Techniques 40
6.1 Assembling Complex Prompts with PromptBuilder 40
6.2 Extending the Framework - Creating Custom PromptItem Types . . 41
6.2.1 Example: Creating a Constraints Item 41
6.3 Dynamic Prompts - Delayed Formatting and Value Injection 42
6.3.1 Example: A Dynamic GoalDefinition Item 43
6.4 Context Reinforcement: Keeping the Agent on Track 44
6.4.1 Blueprint for a Reinforcement-Aware Agent: 44
6.4.2 Example ReinforcementAwareAgent Logic Snippet: 44
6.5 Specialized Prompt Classes for Smaller Models and Manager Agents . 45
6.5.1 `StrictFormatInstruction` 45
6.5.2 `DelegationExample` 46
6.5.3 `ManagerPromptBuilder` 47

7 The Architect's Room - Advanced Planning and Orchestration 49
7.1 A Critical Look at the ReAct Planner 49
7.2 The Plan-and-Execute Paradigm 50
7.3 Blueprint for a Plan-and-Execute System in FAIR 51
7.3.1 Step 1: Defining the Plan Schema with Pydantic 51
7.3.2 Step 2: Engineering the Plan-Generation Prompt 52
7.3.3 Step 3: Creating the PlanGenerator 54
7.3.4 Step 4: Building the PlanExecutorAgent 55

8 The Crucible - Pre-Execution Plan Validation 58
8.1 The Philosophy of Adversarial Review 58
8.2 The Validation Committee - Roles and Responsibilities 59
8.3 Engineering the CriticAgent Prompt 59
8.4 The ValidatedPlannerRunner - The Orchestrator 60

9 The Agent's Mind - Architecting Advanced Memory Systems 62
9.1 The Two Tiers of Memory in FAIR 62
9.1.1 Short-Term Memory: The Conversational Context 62
9.1.2 Long-Term Memory: The Knowledge Library (RAG) 63
9.2 Blueprint for an Advanced Memory Architecture 63
9.2.1 Scoped and Persistent Memory: Giving Each User Their Own Brain 64
9.2.2 Dynamic Memory: The MemorizationTool 65
9.2.3 Managed Forgetting: Pruning and Time-Based Decay 66

10 The Network - Architecting Agent Communication 67
10.1 Communication Patterns in the FAIR Framework 67
10.1.1 Implicit Communication: Orchestration-Driven Workflows . . 67
10.1.2 Explicit Communication: The In-Memory Communicator . . . 68
10.2 Blueprint for a Distributed Communication Backbone 68
10.2.1 Implement a New AbstractCommunicator 69
10.2.2 Create Communication Tools 69
10.2.3 Build a CommunicatorAwareAgent 70
10.2.4 Standardized Message Format and Agent Discovery 70

11 The Agent's Toolkit: Building and Integrating Tools 71
11.1 Why Tools Matter 71
11.2 What a FAIR Tool Is 72
11.2.1 The Three Fields Every Beginner Should Care About 72
11.3 The Tool Lifecycle in FAIR 73
11.3.1 1. Identify the Capability 73
11.3.2 2. Design the Interface 73
11.3.3 3. Implement the Tool Class 73
11.3.4 4. Register the Tool 74
11.3.5 5. Let the Agent Use It 75
11.4 Why the Description Matters So Much 75
11.5 Complex Inputs: Why JSON Is a Common Pattern 76
11.6 Dependency Injection and Reusable Tools 77

11.7 Built-In Tools as Learning Examples 77
11.8 MCP Tools: Extending FAIR Beyond Local Classes 78
11.8.1 The Big Idea Behind MCP 78
11.8.2 The MCP Module Inside fairlib 78
11.8.3 Two Integration Patterns 79
11.8.4 Why This Matters Architecturally 80
11.9 Best Practices for Tool Builders 81
11.10 Security Notes You Should Take Seriously 81
11.11 Closing Perspective 82

12 Installing FAIR-LLM and Working Through the Demo Repository 83
12.1 The Public Package and the Public Demo Repository 83
12.2 Recommended Environment Setup 84
12.2.1 A Practical Python Recommendation 84
12.2.2 Framework-Only Installation 84
12.2.3 Full Demo Environment Setup 85
12.2.4 Environment Variables and Credentials 86
12.2.5 Optional Dependencies by Demo Family 86
12.3 How to Approach the Demos as a Learner 87
12.4 Foundational Single-Agent Demos 87
12.4.1 `demo_single_agent_calculator.py` 87
12.4.2 `demo_advanced_calculator_calculus.py` 88
12.4.3 `demo_agent_config_export_load.py` 88
12.5 Model Abstraction and Backend-Swapping Demos 89
12.5.1 `demo_huggingface_v4.py` 89
12.5.2 `demo_huggingface_v5.py` 89
12.5.3 `demo_model_comparison.py` 90
12.5.4 `demo_single_agent_calculator_load_balancer.py` 90
12.6 Multi-Agent Coordination Demos 91
12.6.1 `demo_multi_agent.py` 91

12.6.2 `demo_multi_agent_load_balancer.py` 91
12.6.3 `demo_web_search_plot_agent.py` 91
12.7 Knowledge-Grounded and RAG Demos 92
12.7.1 `demo_rag_from_documents.py` 92
12.7.2 `demo_faiss_rag_from_readme.py` 92
12.8 Structured Output and Evaluation Demos 93
12.8.1 `demo_structured_output.py` 93
12.8.2 `demo_committee_of_agents_essay_autograder.py` 93
12.8.3 `demo_committee_of_agents_coding_autograder.py` 93
12.9 User-Facing Application Demos 94
12.9.1 `demo_streamlit_simple.py` 94
12.9.2 `demo_streamlit.py` 94
12.9.3 `run_streamlit_demo.sh` 95
12.10 A Good Learning Order Through the Repository 95
12.11 Final Thought 96

13 The Horizon — Present Power and Future Potential 97
13.1 The FAIR Framework Today - A Flexible Engine 97
13.2 The Path to Production - A Roadmap for Real-World Deployment . . 98
13.3 Final Thought: Building the Future of AI Collaboration 99

14 Addendum: The Agent's Cognitive Data Pipeline 101
14.1 The Components of the Pipeline 101
14.2 The Data Pipeline: A Step-by-Step Flow 102
14.3 Visualizing the Data Pipeline 105
14.4 Practical Implementation Notes 105

0.1 Introduction: From Language Models to Autonomous Agents

Large Language Models have changed what many developers expect from software. A program no longer has to be limited to fixed menus, rigid workflows, or hand-authored decision trees. With the right architecture, a model can help a system interpret a user's intent, reason through a task, choose a next step, and adapt as new information arrives. That shift is what makes agentic development worth learning.

An **agentic system** is more than a chatbot with a nice interface. It is a software system that combines language-model reasoning with memory, tool use, and control flow. A well-designed agent can inspect context, decide when it needs help from a calculator, a search system, a retriever, a database, or an external service, and then continue reasoning after it receives the result. That is a very different engineering problem from prompting a model once and printing the reply.

This guide is about learning that engineering discipline in a practical way. Our focus is the **FAIR (Flexible, Agnostic, Interoperable, Reasoning) Framework**, a modular Python framework for building agentic applications. FAIR gives you reusable components for planning, prompting, memory, tool execution, retrieval, orchestration, and multi-agent collaboration. Just as important, it gives you a consistent mental model. Once you understand how the pieces fit together, you can move from a small single-agent demo to a grounded RAG system, a worker committee, or a more advanced planning architecture without starting over each time.

This document is written for students, newcomers to agentic development, and intermediate programmers who want more than a surface-level explanation. It does not assume that you already speak the language of planners, retrievers, vector stores, or agent orchestration. When a concept matters, the guide slows down and explains *why* it matters, what role it plays, and how it appears in the FAIR code and demos. The goal is not merely to show you what classes to import. The goal is to help you build sound instincts about how agentic systems are actually put together.

0.2 What You Are Really Learning Here

As you work through this guide, you are learning several things at once:

- how to think about an agent as a **system**, not just a model call;
- how to separate responsibilities across planners, prompt builders, memory, tool registries, executors, and model adapters;
- how to ground model behavior with retrieval, structured prompts, and explicit execution loops;

- how to decide when a task calls for one agent, several specialized agents, or a different planning strategy altogether; and
- how to connect FAIR to capabilities outside the local Python process, including modern tool ecosystems such as **MCP (Model Context Protocol)**.

That last point deserves special attention. FAIR tools are not limited to code that you write directly inside the framework. FAIR can also be used with MCP-backed tool servers, which means an agent can work with tools exposed by other services over transports such as `stdio` or `SSE`. In practice, that gives you a clean path from local, in-process tool classes to external capabilities such as filesystem access, web search, or domain-specific services running elsewhere. You will see both styles in this guide: traditional FAIR local tools and MCP-based tool access as demonstrated in the companion demos.

0.3 How to Use This Guide

There are at least three good ways to read this document.

- **If you are new to agentic development,** read Chapters 1 through 3 in order. They establish the mental model you need before the deeper reference material makes sense.
- **If you already write Python comfortably but are new to FAIR,** read Chapters 1 and 2, then jump to Chapters 5, 6, and 11 with the demo scripts open beside you.
- **If you are extending the framework itself,** treat this guide as both a tutorial and a reference. Read the chapter introductions for context, then use the deeper chapters to understand the code organization and extension points.

You will get the most out of the guide if you pair it with the companion demo code. FAIR is easiest to learn when you move back and forth between explanation and example. Read a section, find the corresponding demo, and look for the same pattern in real code. You do not need to memorize every class or every file. You do need to become comfortable recognizing the recurring architectural shapes: planner plus registry plus executor plus memory, a retriever behind a knowledge tool, a manager delegating to workers, a schema driving structured output, or a registry being expanded with MCP tools.

0.4 A Roadmap Toward Understanding

This guide is organized to move from foundational ideas to increasingly capable architectures. The chapters are meant to build confidence as you go. Early chapters explain what FAIR is doing. Later chapters show you how to extend it, inspect it, and make more deliberate architectural choices.

Here is the path we will follow:

- **Chapter 1: The FAIR Agentic Framework Explained:** an approachable overview of what an agent is, why a framework helps, and how FAIR organizes the major moving parts.
- **Chapter 2: Under the Hood – How a FAIR Agent Thinks and Acts:** a close look at the core reasoning loop, including how the planner, memory, prompt builder, and executor cooperate during a run.
- **Chapter 3: Advanced Capabilities – From Single Agents to Collaborative Systems:** practical patterns for multi-agent teamwork, RAG, structured output, and secure execution-minded design.
- **Chapter 4: The Expert's Guide – Building a Self-Diagnosing Agent:** an advanced case study that pushes the framework toward reflective and diagnostic workflows.
- **Chapter 5: Core Framework Modules (A Deep Dive):** a guided walk through the codebase so that directories and files stop feeling abstract and begin to feel navigable.
- **Chapter 6: The Prompt Engineering Workbench – Advanced Techniques:** a detailed look at how prompts are composed in FAIR and how to shape behavior with stronger prompt construction.
- **Chapter 7: The Architect's Room – Advanced Planning and Orchestration:** a study of when the default ReAct loop is enough and when a more deliberate planning architecture is the better choice.
- **Chapter 8: The Crucible – Pre-Execution Plan Validation:** a pattern for validating plans before execution, especially when the cost of a poor plan is high.
- **Chapter 9: The Agent's Mind – Architecting Advanced Memory Systems:** deeper memory design, including short-term history, longer-lived retrieval, and strategies for managing context over time.
- **Chapter 10: The Network – Architecting Agent Communication:** communication models for coordinated agents, from simple orchestration to more distributed messaging ideas.

- **Chapter 11: The Agent's Toolkit:** how to build tools well, how to describe them so planners use them correctly, and how FAIR-style local tools compare with MCP-enabled tool ecosystems.
- **Chapter 12: The Horizon – Present Power and Future Potential:** a synthesis of where FAIR is already strong and where it can continue to mature as a more operational framework.
- **Addendum: The Agent's Cognitive Data Pipeline:** a technical walk-through of how information moves among the `SimpleAgent`, planner, prompt builder, memory, and model during execution.

Whether you are building your first tool-using assistant, studying how agentic systems are composed, or preparing to architect larger multi-agent workflows, this guide is meant to give you both confidence and traction. FAIR is not just a place to experiment. It is a place to learn good architectural habits while you experiment.

Chapter 1

The FAIR Agentic Framework Explained

Welcome to FAIR. If you are new to agentic development, this is the right place to start. The goal of this chapter is to make the framework feel approachable before it feels technical. We will clarify what an agent is, why frameworks matter, how FAIR is organized, and how the companion demos fit into the bigger picture. By the end of the chapter, you should have a working mental model of what you are building when you build with FAIR.

1.1 What Is an AI Agent? (And Why You Need a Framework)

A plain chat application sends a prompt to a model and returns the model's reply. That can be useful, but it is limited. An **AI agent** is a larger system that wraps the model in a decision-making loop. Instead of responding once and stopping, the system can inspect context, reason about what to do next, perform an action, observe the result, and then continue.

That shift matters because many real tasks are not solved in one step. Sometimes the model needs to calculate something precisely. Sometimes it needs to search for current information. Sometimes it needs to consult a knowledge base, read a file, or ask another specialized agent to handle part of the work. An agentic system gives the model a structured way to do those things.

At a high level, an agent can:

- **Perceive:** take in a user's request and any relevant context;
- **Reason:** decide what the next useful step should be;
- **Act:** use a tool or delegate work;
- **Observe:** inspect the result of that action; and

- **Continue or finish:** either take another step or produce a final answer.

Once you start building this kind of loop yourself, you quickly discover that most of the difficulty is not in calling the model. The difficulty lies in all the surrounding machinery: keeping history in a usable form, exposing tools, parsing model outputs, handling failures, structuring prompts, and keeping the whole system modular enough that you can change one piece without breaking the rest.

That is why frameworks matter. A good framework does not remove engineering. It organizes it. FAIR gives you a set of reusable parts that make agentic systems far easier to reason about, extend, and teach.

1.2 Introducing the FAIR Framework

The FAIR Agentic Framework is a Python-based system for building modular, tool-using, reasoning-oriented agents. In practice, FAIR revolves around a small set of reusable primitives: model adapters, planners, prompt builders, memory objects, tool registries, tool executors, and agent runners. These parts can be composed in different ways to support very small demos as well as more ambitious systems.

The name FAIR captures the framework's core design posture:

- **Flexible:** components are modular and meant to be recombined. You can add tools, swap planners, change model providers, or alter memory behavior without rewriting the entire stack.
- **Agnostic:** FAIR does not lock you into one language-model vendor. Through the Model Abstraction Layer (MAL), the same agent architecture can be used with OpenAI, Anthropic, Ollama, Hugging Face models, and other adapters.
- **Interoperable:** the framework is built around clear interfaces and shared message types, which makes it easier for planners, tools, memory, and orchestration logic to work together consistently.
- **Reasoning:** FAIR is built around explicit reasoning loops such as ReAct, which means the framework treats multi-step problem solving as a first- class concern rather than an afterthought.

For students and newer developers, FAIR offers a second benefit that is easy to overlook: it gives repeated architectural shape to different kinds of projects. A calculator demo, a RAG demo, a worker committee, and a structured-output workflow all feel related because they are built from the same family of parts. That consistency makes the framework much easier to learn.

1.2.1 Why FAIR Is Friendly to New Builders

A framework becomes beginner-friendly not because it hides everything, but because it introduces complexity in a controlled way. FAIR helps in several practical ways:

- **Shared architecture across projects:** once you understand the basic loop of planner plus registry plus executor plus memory, you can recognize the same pattern in many demos.
- **Clear extension points:** tools, retrievers, prompt components, adapters, and planners all live behind explicit interfaces or recognizable component boundaries.
- **A usable public API:** the `fairlib` import surface exposes the main components you will use most often, which lowers the barrier to getting a first prototype running.
- **Companion demos as executable examples:** the demo set does not merely show what the framework can do. It shows how the parts are assembled in real code.

1.3 The Anatomy of a FAIR Agent

The easiest way to understand FAIR is to think in terms of roles played by the major modules.

- **The brain: planning and prompting.** The planner decides the next step. In the default setup, that means a `ReActPlanner` or `SimpleReActPlanner` calling an LLM and turning the result into either a `(Thought, Action)` pair or a `FinalAnswer`. Prompt construction is handled by the prompting layer, especially `PromptBuilder` and the reusable prompt items defined in `core/prompts.py`.
- **The model interface: the MAL layer.** FAIR uses message-oriented adapters so that the rest of the framework can talk to a common abstraction instead of provider-specific SDKs. This is what makes it possible to change the model backend without redesigning the agent.
- **The hands: tools and execution.** Tools give the agent ways to act on the world. A tool has a name, a description, and a `use()` method. The `ToolRegistry` holds the available tools, and the `ToolExecutor` looks them up and runs them. Built-in examples in the codebase include `SafeCalculatorTool`, `WebSearcherTool`, `WeatherTool`, `GraphingTool`, `KnowledgeBaseQueryTool`, `GradeEssayFromRubricTool`, `GradeCodeFromRubricTool`, and `CodeExecutionTool`.

- **The memory: short-term and retrieval-oriented context.** `WorkingMemory` stores the current conversation history. When longer interactions need compression, `SummarizingMemory` provides an alternate strategy. For grounded retrieval workflows, long-term knowledge can be stored through vector-store-backed components such as `ChromaDBVectorStore` and `FaissVectorStore`.
- **The senses: preprocessing and perception.** FAIR includes explicit extension points for preprocessing and input handling, such as `TextParser` and `EchoPreprocessor`. These are reminders that an agent does not have to accept raw user text without any preparation.
- **The immune system: security and validation.** `BasicSecurityManager` provides a simple validation hook, and the tool executor can use it before sending model-generated input into a tool. This is useful, but it is not a substitute for real operational isolation. Sensitive tools still require careful deployment choices.

1.4 Understanding MCP and Why It Matters

As agentic systems become more useful, they also become more connected. A real agent may need access to files, search, databases, APIs, lab data, internal services, or organization-specific tools that were not written inside the same Python project. That raises an important design question: how should those external capabilities be exposed to the agent in a clean, reusable, and standardized way? One increasingly important answer is **MCP**.

1.4.1 What MCP Is

MCP stands for **Model Context Protocol**. In general terms, MCP is an open standard for connecting language-model applications to external systems in a consistent way. Instead of inventing a one-off integration every time you want an agent to talk to a filesystem, a database, a search service, or a custom tool server, MCP provides a shared protocol for exposing those capabilities.

For a new developer, the simplest way to think about MCP is this: it gives AI applications a common way to discover and use outside tools and context. In the same way that a standard electrical plug or a standard USB port makes it easier to connect devices, MCP makes it easier to connect agents to useful external services without rebuilding that connection pattern from scratch for every project.

This matters because standardization lowers friction. When a protocol becomes widely adopted, developers do not have to keep learning a completely different integration style for every vendor, tool provider, or application stack. They can instead learn the protocol once and then reuse that mental model across many systems. That is one

reason MCP is worth learning early: it is increasingly being treated as an **industry standard** for exposing tools and context to LLM-based applications.

1.4.2 The Basic MCP Mental Model

At a high level, MCP introduces a client-server style relationship.

- An **MCP server** exposes capabilities such as tools, resources, or other useful interfaces.
- An **MCP client** connects to that server and makes those capabilities available to an application.
- The **agent framework**—in this case FAIR—can then present those capabilities to the planner as things the agent may use while solving a task.

That means a file reader served by a local process, or a search capability served over the network, can be surfaced to the agent in a more standardized way. The demos included with FAIR illustrate this with transports such as `stdio` and `SSE`. You do not need to memorize those transport details yet. The important beginner insight is that MCP gives the framework a well-defined bridge to external capabilities.

1.4.3 How MCP Fits into FAIR

FAIR tools do not have to come only from local Python classes. FAIR still works very well with native, in-process tools such as a calculator or a custom Python utility. Those are often the best way to start because they are easy to inspect, modify, and debug. MCP extends that picture rather than replacing it.

In the companion demos, MCP-backed registries expose externally hosted or out-of-process tools to the same overall agent workflow. From the agent's point of view, a filesystem tool served over `stdio` or a search service exposed over `SSE` still appears as an available tool with a name and a description. The planner can decide to use it, the executor can run it, and the agent can observe the result just as it would with a local FAIR tool.

This is a powerful idea for students to absorb early. FAIR is not limited to "whatever tool classes happen to live in your current project." It can work with local tools, composite registries, retrieval systems, and MCP-enabled tool ecosystems. That makes the framework much more realistic for modern agentic engineering, where useful capabilities often live outside the immediate codebase.

1.5 How It Works: The ReAct Cycle in Action

The default FAIR execution pattern is the **Reason + Act** loop, usually called **ReAct**. In plain terms, the agent thinks about the next step, performs an action, inspects the result, and repeats until it has enough information to answer.

Consider a simple request from the calculator demo:

User: "What is (100 + 50) / 25?"

1. **Plan.** The planner sees that the task requires exact arithmetic and decides to use `safe_calculator`.
2. **Act.** The planner emits an `Action` naming the tool and giving it the expression as input.
3. **Execute.** The `ToolExecutor` finds the calculator in the registry and calls its `use()` method.
4. **Observe.** The calculator returns a string result such as "The result of '(100 + 50) / 25' is 6.0". The agent stores that as an observation in memory.
5. **Plan again.** Now the planner sees the observation and recognizes that it has what it needs. It emits a `FinalAnswer`.
6. **Respond.** The agent returns the final text to the user.

This loop is simple enough to understand quickly, but powerful enough to support many useful workflows. It is also the foundation on which more advanced FAIR patterns are built. Multi-agent systems, retrieval, structured output, and even MCP-backed tools still depend on this core rhythm of reasoning, action, and feedback.

1.6 Getting Started: Your First FAIR Agent

The best way to start with FAIR is to run a small demo, then read the code with this chapter in mind. The calculator demos are especially good first stops because they expose the basic architecture without too many moving parts.

A typical first agent setup follows a recognizable pattern:

1. choose a model adapter;
2. create a `ToolRegistry` and register one or more tools;
3. create a `ToolExecutor`;

4. create a memory object such as `WorkingMemory`;
5. create a planner using the model and registry; and
6. assemble everything into a `SimpleAgent`.

You will see this pattern in the demo scripts again and again. That is not an accident. Repetition is how the framework teaches you its architecture.

1.6.1 A Minimal First Example

```python
from fairlib import (
    HuggingFaceAdapter,
    ToolRegistry,
    ToolExecutor,
    WorkingMemory,
    SimpleAgent,
    SimpleReActPlanner,
    SafeCalculatorTool,
)

llm = HuggingFaceAdapter("dolphin3-qwen25-3b")

registry = ToolRegistry()
registry.register_tool(SafeCalculatorTool())

executor = ToolExecutor(registry)
memory = WorkingMemory()
planner = SimpleReActPlanner(llm, registry)

agent = SimpleAgent(
    llm=llm,
    planner=planner,
    tool_executor=executor,
    memory=memory,
    max_steps=5,
)
```

That small block of code captures the heart of FAIR. Once you understand it, new capabilities feel like additions to the same architecture rather than a completely different framework.

1.6.2 How to Learn Faster from the Demos

As you work through the companion demos, ask the same questions each time:

- What tools are available to the agent?
- Which planner is being used, and why?
- Is the agent stateful or stateless?
- What role does memory play in this example?
- Is the agent acting alone, using retrieval, or collaborating with other agents?

That habit will help you move from copying examples to understanding design. FAIR becomes much easier once you recognize the recurring structure beneath the surface differences between demos.

1.7 What Comes Next

This chapter introduced the major pieces of the framework at a conceptual level. The next chapter slows down and traces the actual execution flow in more detail. That is where the framework starts to feel less like a list of classes and more like a living system.

Chapter 2

Under the Hood - How a FAIR Agent Thinks and Acts

Chapter 1 gave us the architectural map. This chapter follows the actual path a request takes through the framework. If Chapter 1 answered, "What are the main parts of a FAIR agent?" this chapter answers, "What happens when those parts start working together?"

For newer developers, this is an important turning point. Agentic systems can feel mysterious when viewed only from the outside. Once you watch a request move through memory, prompting, planning, tool execution, and observation, much of that mystery disappears. The system starts to feel inspectable. That matters, because inspectable systems are easier to debug, extend, and trust.

As you read, it can help to keep the corresponding code open beside you, especially `modules/agent/simple_agent.py`, `modules/planning/react_planner.py`, and `core/prompts.py`. This chapter is not just a theory chapter. It is meant to make the code easier to read.

2.1 The Core Loop: A Walk Through SimpleAgent.arun()

The entire life of an agent's task happens within the arun method of the SimpleAgent class (found in modules/agent/simple_agent.py). This method orchestrates the **ReAct (Reason + Act)** cognitive cycle. Let's walk through its execution flow.

When you call agent.arun("your query"), this is what happens:

1. **Memory Check:** For "stateless" agents (often used as workers in a multi-agent team), the agent first clears its WorkingMemory. For a typical conversational agent, it preserves the history.

2. **History Retrieval:** The agent asks its WorkingMemory (modules/memory/base.py) for the current conversation history. This history is a list of Message objects (core/message.py).
3. **The Critical Step: Calling the Planner:** The agent passes the entire history and the current user request to its assigned **Planner** by calling planner.aplan(history, user_input). This is the moment the agent "thinks."
4. **Decision Point:** The Planner returns one of two things:
   - A FinalAnswer: If the Planner believes the task is complete, the arun loop terminates and the answer is returned to the user.
   - A (Thought, Action) tuple: This is the core of the ReAct cycle. The agent has reasoned about what to do next.
5. **Execution:** The SimpleAgent takes the Action object and passes it to the ToolExecutor.
6. **Observation:** The ToolExecutor runs the specified tool and returns a string result. The SimpleAgent wraps this result in a standard format: Message(role="system", content=f"Observation: {result}").
7. **Memory Update:** The agent's Thought (as an assistant message) and the resulting Observation (as a system message) are added back to WorkingMemory.
8. **Loop:** The process repeats, but this time the Planner has the previous turn's Thought and Observation as context to inform its next decision.

This loop continues until the Planner decides to output a FinalAnswer or a maximum step limit is reached.

2.2 The Planner: The Agent's "*Brain*"

The Planner (modules/planning/react_planner.py) is the most critical component for shaping an agent's behavior. Its sole purpose is to generate the next (Thought, Action) pair by calling a Large Language Model. The FAIR framework provides two default planners to suit different models:

1. ReActPlanner (for powerful models):
   - Uses a sophisticated prompt that requires the LLM to output its decision as a structured JSON object.
   - Best For: High-end models like GPT-4, Claude 3 Opus, or other large, instruction-following models that reliably adhere to complex formatting rules.

2. SimpleReActPlanner (for smaller/local models):
    - Uses a simplified prompt that asks the LLM for a basic key-value pair format instead of strict JSON.
    - Best For: Smaller, open-source models that may struggle with generating perfect JSON.

While the planners contain logic for calling the LLM and parsing its response, their "personality" and reasoning ability are defined almost entirely by their prompts. This brings us to the heart of the framework's "magic."

2.3 The Art of Prompt Engineering: PromptBuilder and Prompt Items

Many frameworks treat prompts as simple f-strings. This quickly becomes messy and unmanageable. **The FAIR framework's philosophy is that prompt engineering is a first-class architectural concern.** A well-structured prompt leads to a reliable and predictable agent.

Prompt construction is handled by `PromptBuilder`, and all reusable prompt items — `RoleDefinition`, `ToolInstruction`, `FormatInstruction`, `Example`, `History`, and more — are defined together in `core/prompts.py`. This module also contains the abstract `PromptItem` base class that all items implement. Specialized subclasses including `StrictFormatInstruction` (explicit DO/DON'T formatting rules for smaller 3–7B models), `DelegationExample` (single-turn delegation examples for manager agents), and `ManagerPromptBuilder` (a `PromptBuilder` subclass with built-in strict mode and workflow ordering) are also defined there.

2.3.1 Anatomy of a Prompt

The PromptBuilder combines several types of PromptItem to construct the final system prompt that gets sent to the LLM:

- **RoleDefinition**: This is the highest-level instruction. It tells the agent *who it is* and its overall goal (e.g., "You are an expert mathematical calculator.").
- **ToolInstruction**: The builder automatically generates one of these for each tool registered in the ToolRegistry. It tells the LLM the tool's name and description so it knows what it can do.
- **FormatInstruction**: This is the most rigid part of the prompt. It provides explicit, non-negotiable rules about how the LLM should structure its response (e.g., "Your response MUST be a single, valid JSON object.").

- **Example**: This provides few-shot examples of a perfect thought/action process. This is one of the most powerful ways to guide the LLM's reasoning, especially for multi-step tasks.
-

2.3.2 The "Magic" Revealed: Deconstructing a Default Prompt

Let's look at the default prompt builder for the `SimpleReActPlanner` (`_create_default_simple_kv_builder` in `modules/planning/react_planner.py`).

```
# --- YOUR RESPONSE FORMAT (MANDATORY) ---

On every turn, you MUST provide your reasoning in a
'Thought' and your action in an 'Action' block.

Your response must use this exact key-value format:

Thought: [Your analysis and step-by-step reasoning for what
    to do next.]

Action:

tool_name: [The name of the ONE tool to use, or 'final_answer
    ']

tool_input: [The input for that tool, or the complete final
    answer for the user.]

# --- YOUR REASONING PROCESS ---

1. First, look at the user's request to understand their
    ultimate goal.

2. Next, look at the most recent 'system' message in the
    history. This
   is the 'Observation' from your last action.

3. If this Observation contains the complete answer to the
    user's goal,
   your action MUST be to use the 'final_answer' tool.

4. If the Observation is not enough, you MUST choose another
    tool...
```

Why is this so specific?

- **Alignment with Code:** The instruction "look at the most recent 'system' message" is not arbitrary. It perfectly aligns with the SimpleAgent's code, which, as we saw, wraps tool outputs in Message(role="system", ...). This tight coupling between the prompt's instructions and the framework's implementation is a key design principle that dramatically increases reliability.
- **Reducing "Hallucination":** By providing a strict format and clear examples, the prompt leaves very little room for the LLM to "get creative" and generate conversational text when the planner is expecting a structured command.
- **Predictability:** This structure ensures that the planner can reliably parse the LLM's output using regular expressions or JSON parsers, making the agent's behavior predictable.

2.4 Developer Customization: How to Change Your Agent's "Personality"

Because the prompt is built from modular components, you have granular control over the agent's behavior. The most common customization point is the **Planner's PromptBuilder**.

You can access it directly after creating a planner: planner.prompt_builder.

2.4.1 Changing the Agent's Role (The Easiest & Most Common Override)

You can give your agent a completely new persona by simply replacing its RoleDefinition. This is demonstrated in demo_advanced_calculator_calculus.py:

```
# In demo_advanced_calculator_calculus.py
# Create the planner with its default prompt
planner = SimpleReActPlanner(llm, tool_registry)
# ----> THE OVERRIDE <----
# Replace the default role with a new, more specific one.
planner.prompt_builder.role_definition = \
    RoleDefinition(
        "You are an advanced expert mathematical calculator whose job it is to "
        "perform calculations.\n"
        "You must reason step-by-step to determine the best course of action. "
        "If a user's request requires "
```

```
        "multiple steps or tools, you must break it down and execute them "
        "sequentially.")
```

2.4.2 Teaching the Agent New Tricks (Changing the Examples)

If you are building an agent for a complex task that requires a specific sequence of actions, the best way to teach it is by example. You can clear the default examples and add your own.

```
# Clear existing examples
planner.prompt_builder.examples.clear()
# Add a new, custom example
planner.prompt_builder.examples.append(
    Example(
        "# --- Example: Custom Multi-Step Workflow ---\n"
        "user: Find the price of Bitcoin and tell me the weather in that city.\n"
        "assistant: "
        "Thought: First, I need to find the price of Bitcoin to know what "
        "'that city' refers to. I will use the web_searcher tool.\n"
        "Action:\n"
        "tool_name: web_searcher\n"
        "tool_input: current price of Bitcoin\n"
        "\n"
        "system: Observation: The current price of Bitcoin is $65,000 USD in "
        "San Francisco.\n"
        "assistant: "
        "Thought: I have the price and the city (San Francisco). Now I need to "
        "find the weather there. I will use the weather tool.\n"
        "Action:\n"
        "tool_name: weather\n"
        "tool_input: weather in San Francisco"
    )
)
```

2.5 From Thought to Action: The ToolExecutor

Once the Planner produces an Action, the SimpleAgent passes it to the ToolExecutor. The ToolExecutor.execute() method from modules/action/executor.py is simple but robust. It's responsible for finding the tool in the ToolRegistry, (optionally) validating input with the SecurityManager, running the tool's .use() method, and handling errors gracefully.

2.5.1 Completing the Loop: Memory and Observation

The ToolExecutor's output (the result) is wrapped in an "Observation" message. The SimpleAgent creates this observation message (Message(role="system", content=f"Observation: {str(observation_output)}")) and adds it back to WorkingMemory. This "Observation" becomes the key piece of new information for the Planner in the *next* loop iteration, allowing the agent to react to the results of its actions.

Chapter 3

Advanced Capabilities - From Single Agents to Collaborative Systems

Once you understand the single-agent loop, FAIR starts to open up in exciting ways. The framework is not limited to one model, one tool, and one answer path. It can support teams of specialized agents, retrieval-grounded reasoning, validated structured output, and execution patterns that are much closer to the needs of real applications.

This chapter introduces several of those higher-leverage patterns. It is meant to feel practical. These are not abstract ideas added for completeness. They are patterns you are likely to use when a basic single-agent assistant stops being enough.

As you move through the sections, pay attention to the reason each pattern exists. FAIR does not add complexity just to be clever. Each advanced pattern is a response to a real limitation: one agent is not always the right agent, base-model knowledge is not always the right source of truth, free-form text is not always the right output format, and unrestricted execution is never the right security posture.

3.1 Multi-Agent Collaboration: The "Committee of Agents"

3.1.1 Philosophy:

A single, general-purpose agent, no matter how powerful the underlying LLM, will eventually fail when faced with a sufficiently complex, multi-domain task. The principle of separation of concerns is as vital in AI engineering as it is in traditional software engineering. Instead of creating one monolithic "*God agent*," the FAIR framework encourages building a team of specialized agents, each with a narrow role and a limited set of tools. This approach, outlined in the ***Agentic System Design Blueprint.pdf***, leads to systems that are more robust, easier to debug, and more reliable.

3.1.2 Technical Implementation:

This pattern is primarily implemented by three key components, which you can see in action in demo_multi_agent.py and the more complex autograder scripts.

- **The Orchestrator (HierarchicalAgentRunner):** Found in modules/agent/-multi_agent_runner.py, this class is the master conductor. It does not have its own reasoning logic; instead, it facilitates the conversation between a Manager and its Workers. The arun() method manages the entire workflow. Its core responsibility is to:

  1. Pass the initial user request to the Manager agent.
  2. Receive a "delegate" action from the Manager.
  3. Find the specified Worker agent from its dictionary of workers.
  4. Invoke the Worker agent with the delegated task.
  5. Wait for the Worker to complete its internal ReAct loop and return a final answer.
  6. Package the Worker's answer into a standardized "Observation" message and pass it back to the Manager for its next planning step

- **The "Brain" of the Manager (ManagerPlanner):** This specialized planner, found in modules/agent/multi_agent_runner.py, is what makes the hierarchical system work. Its prompt, built by _create_default_manager_prompt_builder(), is fundamentally different from a standard ReActPlanner prompt:

  – **It has no direct tools:** Its prompt does not instruct it to use tools like calculators or web searchers. The list of available tools is intentionally omitted.

  – **Its only action is delegation:** The examples and format instructions are engineered to make the LLM output one of two actions: delegate (to a worker) or final_answer. The delegate action is not a real "tool" in a registry; it's a special command interpreted directly by the HierarchicalAgentRunner.

  – **It thinks about team strategy:** The prompt forces the manager to reason about *who* on its team is best suited for a sub-task. It does this by receiving the list of workers and their role_description attributes, which are injected directly into the prompt, giving it a manifest of its team's capabilities.

- **The Workers (SimpleAgent):** The workers in this pattern are typically "stateless" SimpleAgent instances. They are configured with a very narrow RoleDefinition and a limited set of specialized tools, allowing them to excel at their specific function without being distracted by irrelevant capabilities.

Let's trace the workflow from `demo_multi_agent.py` where the user asks: "My budget is $5,000. Please find the current price of Bitcoin and then calculate exactly how many Bitcoins I can afford to buy."

1. **Manager's First Turn:** The HierarchicalAgentRunner passes the query to the ManagerAgent. Its ManagerPlanner sees the two-part request.
   - **Thought:** "I need to find the price of Bitcoin first. The ' Researcher'agent is described as a specialist for finding real-time information. I will delegate this task to it."
   - **Action:** `{"tool_name": "delegate", "tool_input": {"worker_name": "Researcher", "task":`
2. **Researcher's Turn:** The HierarchicalAgentRunner invokes the Researcher agent. The Researcher, equipped only with the WebSearcherTool, runs its own internal ReAct loop and returns the answer: "According to top search results, the current price of Bitcoin is $65,000 USD."
3. **Manager's Second Turn:** The runner packages the researcher's output into an observation for the manager.
   - **History now contains:** system: Observation: Result from Researcher: According to top search results, the current price of Bitcoin is $65,000 USD.
   - **Thought:** "I have the price. Now I need to perform the calculation. The ' Analyst'agent is described as a specialist for mathematical calculations. I will delegate the calculation to it."
   - **Action:** `{"tool_name": "delegate", "tool_input": {"worker_name": "Analyst", "task": "C`
4. **Analyst's Turn:** The Analyst agent is invoked. It uses its SafeCalculatorTool and returns the result: "The result of '5000 / 65000' is 0.0769..."
5. **Manager's Final Turn:** The manager receives the final piece of information.
   - **History now contains:** system: Observation: Result from Analyst: The result of '5000 / 65000' is 0.0769...
   - **Thought:** "I now have all the information required to answer the user's original request. I will synthesize the findings into a final answer."
   - **Action:** {"tool_name": "final_answer", "tool_input": "Based on a budget of $5,000 and a Bitcoin price of $65,000, you can afford approximately 0.0769 Bitcoins."}

This structured delegation ensures that the right agent performs the right task, leading to a more reliable and accurate outcome than a single agent could achieve.

3.2 Grounding Agents in Reality: Advanced RAG

3.2.1 Philosophy:

An LLM's knowledge is static and general. Retrieval-Augmented Generation (RAG) is the process of providing the LLM with specific, timely, and relevant context to "ground" its response in a source of truth. This is the single most effective technique for mitigating factual "hallucinations" and making agents experts on specific, private, or up-to-date information.

3.2.2 The Technical RAG Pipeline:

FAIR supports two practical RAG paths. The lighter ChromaDB path is demonstrated in `demo_rag_from_documents.py`, while a more production-oriented FAISS path with optional reranking is demonstrated in `demo_faiss_rag_from_readme.py`.

Phase 1: Ingestion (Building the Knowledge Base)

This is the offline process of preparing your knowledge for the agent.

1. 1. Loading + Extraction (`DocumentProcessor`): The current framework uses `utils/document_processor.py` as the main ingestion utility. It can extract text from PDF, DOCX, PPTX, XLSX, TXT, CSV, SQL, and Markdown-like inputs, and it can optionally OCR images and image-based pages.

2. 2. Chunking (`split_text_semantic`): Instead of a separate `DocumentLoader`, the `DocumentProcessor` now owns the default chunking path. It splits extracted text into `Document` objects with source-aware metadata.

3. 3. Embedding (`SentenceTransformerEmbedder`): Each chunk is converted into a dense vector using a sentence-transformer model.

4. 4. Storing (`ChromaDBVectorStore` or `FaissVectorStore`): ChromaDB is the simpler in-memory/persistent vector-database path. FAISS is the more explicit index-based path and supports persistence plus optional GPU use.

Phase 2: Retrieval (Using the Knowledge Base at Runtime)

This is the online process that happens when the user interacts with the agent.

1. 1. User Query: The user asks a question that requires grounded knowledge.

2. 2. Planning: The planner decides that the agent should use `KnowledgeBaseQueryTool` rather than answer from the base model alone.

3. 3. Execution: The tool calls a retriever. In the basic path this is `SimpleRetriever`; in the FAISS demo the retriever can be wrapped by `CrossEncoderRerankingRetriever` for a second-stage rerank.

4. 4. Retrieval: The retriever embeds the query, performs similarity search, and returns the most relevant chunks or `Document` objects.

5. 5. Observation: The tool formats the retrieved text into an observation string and returns it to the agent loop.

6. 6. Generation: The planner runs again with grounded context now present in memory, allowing the LLM to synthesize a better final answer.

3.3 Enforcing Structure: Reliable JSON Output and Pydantic Validation

3.3.1 Philosophy:

For reliable systems, you cannot trust an LLM to "probably" return data in the right format. You must compel it. This is essential for any task involving data extraction, API calls, or interactions that require a predictable, machine-readable output.

3.3.2 Technical Implementation (demo_structured_output.py):

The framework demonstrates a powerful and robust pattern for achieving this, centered on Pydantic and a self-correction loop.

- **Step 1: Define the Schema with Pydantic:** You first define your target data structure as a Pydantic BaseModel. This class, like UserProfile in the demo, acts as the canonical schema, defining the exact fields, data types, and descriptions for the data you want to extract.

```
class UserProfile(BaseModel):
    name: str = Field(..., description="The full name of the user.")
    age: int = Field(..., description="The age of the user.")
    interests: List[str] = Field(..., description="A list of the user's hobbies.")
```

- **Step 2: Engineer a Hyper-Specific Prompt:** A generic prompt will not work. You must create a prompt that acts as a strict contract with the LLM. The EXTRACTION_PROMPT_TEMPLATE in the demo is a masterclass in this, containing several key instructions:
  - **Role Definition:** "You are a highly efficient information extraction assistant."
  - **Negative Constraints:** "Do NOT include any commentary, explanation, or extra text."
  - **Format Rules:** "Only return the raw JSON object—no Markdown, no preamble."
  - **Schema Injection:** The prompt dynamically includes the JSON schema generated from your Pydantic model, telling the LLM the exact structure it must produce.
- **Step 3: The Self-Correction Loop:** This is the most critical part of the pattern. The ExtractionAgent in the demo doesn't just call the LLM once; it tries, validates, and retries on failure.

```python
# Inside the agent's extraction method...
for attempt in range(max_retries):
    try:
        # Call the LLM with the engineered prompt
        response_text = self.llm.chat(...)
        # Attempt to validate the response against the Pydantic model
        validated_output = UserProfile.model_validate_json(response_text)
        # If successful, return the validated data
        return validated_output
    except ValidationError as e:
        # If validation fails, the Pydantic error is a perfect
        # piece of feedback for the LLM.
        print(f"Validation Failed: {e}")
        # Append the error to the prompt and retry
        prompt += f"\n\nERROR MESSAGE:\n{e}\nPlease correct the output and try again."
```

When validation fails, the ValidationError from Pydantic provides a detailed, machine-readable explanation of what went wrong (e.g., "Field ' age'is not a valid integer"). By feeding this precise error message back to the LLM in the next attempt, the agent asks the model to correct its own mistake, dramatically increasing the probability of getting valid JSON on the second or third try.

3.4 Practical Security Considerations

3.4.1 Philosophy:

An agent that can execute code (CodeExecutionTool) or access external data sources has a significantly larger attack surface than a simple chatbot. Security cannot be an afterthought; it must be a foundational component of the agent's design.

3.4.2 Technical Implementation:

- **Input Validation (The First Line of Defense):** The BasicSecurityManager in modules/security/basic_security_manager.py provides a first line of defense. It inspects text inputs against a list of regular expression patterns designed to catch basic prompt injection attacks (e.g., "ignore previous instructions"). While not foolproof against sophisticated attacks, it serves as an essential, easy-to-implement safeguard.
- **Secure Execution and Sandboxing (Non-Negotiable):** The framework explicitly warns that the default CodeExecutionTool is **NOT SECURE**. Its use of Python's subprocess module is a functional placeholder for demos. Running subprocess executes commands directly on the host machine, meaning a malicious code submission could access the filesystem, network, or environment variables, posing a severe security risk.

A production-grade implementation **must** replace this with a robust sandboxing technology that provides true isolation. As outlined in InitialNotes.txt and the Agentic System Design Blueprint.pdf, this involves:

- **Containerization (Docker):** Running each piece of code in an isolated, ephemeral Docker container with a restricted filesystem and no network access by default.
- **MicroVMs (gVisor, Firecracker):** For an even higher level of security, these technologies provide a dedicated guest kernel for each execution, creating a strong boundary between the agent's code and the host system.

Note: Implementing a true sandbox is the single most important security measure for any agent that is permitted to execute code.

Chapter 4

The Expert's Guide - Building a Self-Diagnosing Agent

In the previous chapters, we have explored agents that use tools to act upon the world. This chapter pushes further into that concept, guiding you through an expert-level project: building a system of agents that can reason about, generate code for, and diagnose bugs *in other agents*. This is a "*meta-cognitive*" task where the AI system turns its analytical capabilities inward, examining its own kind to find and explain flaws.

This is more than a theoretical exercise. Building a system that can automate bug diagnosis demonstrates a mastery of the FAIR framework's most advanced patterns, particularly the "committee of agents" architecture. It is a testament to the idea that complex problems are best solved not by a single monolithic "God agent," but by a collaborative team of specialists, each excelling at a specific part of the problem.

4.1 The Architectural Blueprint - A Committee of Diagnosticians

The task of diagnosing a bug is not a single action but a multi-stage investigation requiring distinct skills. Therefore, we will architect our solution as a "committee of agents," managed by a lead orchestrator. This pattern is implemented using the HierarchicalAgentRunner and a specialized ManagerPlanner.

The committee consists of the following specialists:

- **DiagnosisManager (The Orchestrator):** This is the team lead. It does not perform any analysis itself; its sole purpose is to understand the overall goal (diagnose the bug) and delegate each step of the investigation to the appropriate specialist worker. It is a SimpleAgent equipped with a ManagerPlanner.
- **HypothesisAgent (The Theorist):** This agent's job is to form a clear, testable hypothesis about the cause of a bug. It receives the initial bug report and must

have the ability to read the relevant source code files to understand the context and pinpoint a likely cause.

- **TestGeneratorAgent (The Coder):** A code-generation specialist. It receives the hypothesis from the HypothesisAgent and its only job is to write a unit test (using a library like pytest) that will either prove or disprove that hypothesis. This requires an extremely precise prompt to ensure it generates valid, executable code.
- **TestExecutorAgent (The Operator):** A secure worker whose only tool is the CodeExecutionTool. It takes the unit test code generated by the TestGeneratorAgent, executes it against the buggy code, and reports the raw results (pass, fail, or error) back to the manager.
- **ReportSynthesizerAgent (The Scribe):** The final agent in the chain. It takes all the artifacts from the investigation—the initial report, the hypothesis, the generated test code, and the execution results—and synthesizes them into a clear, human-readable final diagnosis for the developer.

4.2 Engineering the Tools for Diagnosis

A specialized team needs specialized tools. This system relies on two critical tool categories:

4.2.1 The CodeExecutionTool

This tool is the core of the TestExecutorAgent's capabilities. As implemented in modules/action/tools/code_execution_tool.py, it takes a string of Python code (the unit test) and executes it, returning the output from stdout and stderr.

CRITICAL SECURITY WARNING: As stressed throughout this guide and in the tool's documentation, the default CodeExecutionTool uses a simple subprocess call. This is **NOT SECURE** for production use. Executing untrusted code requires a robust sandboxing solution like Docker, gVisor, or Firecracker to prevent malicious code from harming the host system. [8]

4.2.2 The FileContentTool (Hypothetical)

For the HypothesisAgent to form an educated guess, it must be able to read the source code of the framework. We would need to equip it with a tool, let's call it FileContentTool, that can read the contents of a given file path. This is a straightforward tool to implement and is analogous to the DocumentLoader used in the RAG demos.

4.3 The Master Plan: Orchestration with the DiagnosisManager

The entire diagnostic process is orchestrated by the DiagnosisManager. Its behavior is defined not by Python code, but by the prompt given to its ManagerPlanner. This prompt defines the master plan for the entire operation.

An example of the ManagerPlanner's prompt builder for the DiagnosisManager would be engineered to force delegation in a specific sequence:

```
# In a setup script for the DiagnosisManager:
manager_planner.prompt_builder.role_definition = RoleDefinition(
    "You are a Diagnosis Manager. Your job is to orchestrate a team of "
    "agents to diagnose a bug. You must delegate tasks in a specific "
    "sequence."
)

manager_planner.prompt_builder.examples.append(
    Example(
        """
User Request: [Bug Report] The SimpleAgent is failing with a
timeout when I give it a complex calculus problem.

Thought: The user has reported a bug. My first step is to form a
hypothesis. I will delegate this to the HypothesisAgent.

Action: {"tool_name": "delegate", "tool_input": {"worker_name":
"HypothesisAgent", "task": "Form a hypothesis for why the SimpleAgent
times out on complex calculus problems. Read the source code for
SimpleAgent and AdvancedCalculusTool to inform your hypothesis."}}

Tool Observation: Result from HypothesisAgent: The probable cause is
that the ReAct loop does not have a step to simplify a complex calculus
problem before passing it to the AdvancedCalculusTool. The tool likely
fails on the unsimplified expression.

```

```
Thought: I have a clear hypothesis. Now I need a unit test to verify it.
I will delegate this to the TestGeneratorAgent.

Action: {"tool_name": "delegate", "tool_input": {"worker_name":
"TestGeneratorAgent", "task": "Write a pytest unit test that recreates
the timeout failure by passing a complex, unsimplified calculus problem
to a SimpleAgent equipped with an AdvancedCalculusTool. The test should
assert that the agent's run method fails or times out."}}

Tool Observation: Result from TestGeneratorAgent: [Contains a full
string of Python pytest code]

Thought: I have the test code. Now I must execute it to see if it fails
as expected. I will delegate this to the TestExecutorAgent.

Action: {"tool_name": "delegate", "tool_input": {"worker_name":
"TestExecutorAgent", "task": "Execute this pytest code and return the
full output."}}

Tool Observation: Result from TestExecutorAgent: [Pytest output
showing the test failed as expected with a timeout]

Thought: The test has confirmed the hypothesis. The investigation is
complete. I now have all the necessary components (bug report,
hypothesis, test code, and results) to write the final report. I will
delegate this to the ReportSynthesizerAgent.

Action: {"tool_name": "delegate", "tool_input": {"worker_name":
"ReportSynthesizerAgent", "task": "Synthesize all the information into a
final bug diagnosis report."}}

Tool Observation: Result from ReportSynthesizerAgent: [A full
```

```
    ,
formatted diagnosis report]

Thought: I have the final report. The task is complete.

Action: {"tool_name": "final_answer", "tool_input": "[The final
diagnosis report from the synthesizer]"}
        """
    )
)
```

This example-driven prompt forces the DiagnosisManager to follow the exact workflow we designed, converting a complex debugging problem into a manageable, linear sequence of expert tasks.

4.4 Assembling the Committee in Code

Bringing this system to life uses the same patterns demonstrated in the autograder demos (demo_committee_of_agents_coding_autograder.py and demo_committee_of_agents_essay_autograder.py). We would use a factory function to create our specialized worker agents and then wire them into the HierarchicalAgentRunner.

```python
# A conceptual script for running the diagnosis committee
from fairlib import (
    HierarchicalAgentRunner,
    ManagerPlanner,
    SimpleAgent,
    # ... other necessary imports
)

# Assume 'create_agent' is a helper function like in the autograder demos
# Assume CodeExecutionTool and FileContentTool are defined and imported
# 1. Initialize the LLM
llm = OpenAIAdapter(...)
# 2. Create the worker agents
hypothesis_agent = create_agent(llm, "A diagnostician that forms hypotheses about bugs.", [FileContentTool()])
test_generator_agent = create_agent(llm, "A coding agent that only writes pytest unit tests.", [])
test_executor_agent = create_agent(llm, "A secure execution agent.", [CodeExecutionTool()])
```

```python
report_synthesizer_agent = create_agent(llm, "A technical writer that synthesizes reports.", [])
workers = {
    "HypothesisAgent": hypothesis_agent,
    "TestGeneratorAgent": test_generator_agent,
    "TestExecutorAgent": test_executor_agent,
    "ReportSynthesizerAgent": report_synthesizer_agent,
}

# 3. Create the Manager Agent
manager_memory = WorkingMemory()
# The manager_planner would be configured with the detailed prompt shown in the previous section
manager_planner = ManagerPlanner(llm, workers)
manager_agent = SimpleAgent(
    llm=llm,
    planner=manager_planner,
    tool_executor=None,  # The manager delegates, it does not execute tools
    memory=manager_memory
)

# 4. Create and run the orchestrator
team_runner = HierarchicalAgentRunner(manager_agent, workers)
# 5. Kick off the diagnosis
bug_report = "The agent is failing with a timeout on complex calculus problems."
final_diagnosis = await team_runner.arun(bug_report)
print(final_diagnosis)
```

4.5

4.6 Conclusion: A Framework for Complex Problem-Solving

This chapter has provided a blueprint for one of the most advanced applications of an agentic framework: a system that can diagnose itself. While a full implementation requires careful engineering, especially around security, the architectural pattern is clear and achievable with the FAIR framework.

By using a committee of specialized agents, orchestrated by a manager that follows a clear plan, we can break down an incredibly complex and abstract problem—software debugging—into a series of concrete, manageable steps. This meta-cognitive capability

showcases the true power of multi-agent systems and provides a robust pattern for tackling almost any complex, multi-domain challenge.

Chapter 5

Core Framework Modules (A Deep Dive)

This chapter is the guided walk through the FAIR codebase. It is written for the moment when the framework stops being only an idea and starts becoming a real set of files, modules, and responsibilities in your editor. For students and new contributors, that moment can be intimidating. A codebase can feel much larger than it really is until someone explains how the pieces are grouped and why they exist.

The point of this chapter is not to force you to memorize every file. It is to help you orient yourself quickly. If you know where interfaces live, where the agent loop lives, where prompt assembly happens, where tools are registered, and where retrieval components sit, you can learn the rest much faster.

You should read this chapter as both a map and a reference. Skim it once to get your bearings. Then come back to the relevant sections as you work through the demos or begin extending the framework yourself.

5.1 The Architectural Heart: core/interfaces/

The single most important directory for understanding the framework's power and flexibility is core/interfaces/. This folder contains the architectural DNA of the entire system. Each file defines an Abstract Base Class (ABC), which is not a piece of working code but a "contract" or a "blueprint." Any component that wants to perform a specific role in the framework (like being a planner or a memory system) *must* promise to follow the rules of its corresponding interface. This interface-driven design is the key to achieving the framework's core principles:

- **Flexibility & Modularity**: Because agents interact with the abstract interfaces, not the concrete implementations, you can "unplug" one component and "plug in" another without rewriting the agent's logic. For example, you can swap a simple WorkingMemory for the more advanced SummarizingMemory because both honor the AbstractMemory contract.

- **Agnosticism**: This is most evident with the AbstractChatModel. The ReAct-Planner doesn't know or care if it's talking to OpenAI or a local Hugging Face model; it only knows it's talking to something that fulfills the AbstractChat-Model contract. This prevents vendor lock-in and allows you to choose the best tool for the job.
- **Extensibility**: If you want to create a new tool or connect to a new vector database, you simply need to create a new class that inherits from the appropriate ABC and implements the required methods. The framework will instantly know how to work with your new component.

5.2 The Data of Conversation: core/message.py and core/types.py

Data structures are the lifeblood of the framework, flowing between every component.

- **core/message.py**: This module defines the objects that represent the dynamic, turn-by-turn flow of an agent's reasoning process.
  - **Message**: This is the universal data object for conversation. Every interaction—from the user, to the agent's internal thought, to the tool's output—is encapsulated in a Message object with a specific role (user, assistant, system, tool). This standardized structure is critical for how memory is stored and how prompts are built.
  - **Thought, Action, FinalAnswer**: These are not generic messages; they are the specific, structured outputs of the Planner. They represent the discrete stages of the ReAct cognitive cycle. A Thought is the agent's internal monologue ("what should I do next?"). An Action is its decision ("I will use this tool with this input."). A FinalAnswer is its conclusion ("I'm done."). The SimpleAgent's job is to interpret these specific objects to orchestrate the loop.
- **core/types.py**: This module defines more static, application-level data structures.
  - **Document**: This is the standard representation for a chunk of external knowledge. When you load a file for a RAG pipeline, it's split into Document objects, each containing text (page_content) and metadata.
  - **FinalGrade, GradingCriterion**: These Pydantic models are a perfect example of enforcing structure. By defining the exact JSON schema for a grade report, they allow tools like the GradeEssayFromRubricTool to have a clear target, ensuring reliable, validated, and machine-readable output from the LLM.

5.3 The Brain: The Cognitive and Planning Modules

This is where the agent "thinks." The process involves a tight collaboration between the planner, the prompt builder, and the underlying language model.

- **modules/planning/react_planner.py**: This file contains the concrete implementations of the AbstractPlanner interface.
  - ReActPlanner and SimpleReActPlanner are the default reasoning engines. Their core responsibility is to take the conversation history and generate the next (Thought, Action) pair or a FinalAnswer. The key difference between them lies in the complexity of the output format they expect from the LLM (structured JSON vs. simple key-value pairs), which makes them suitable for different classes of models.

- `core/prompts.py`: The complete prompt engineering system lives in a single module in the core layer. `PromptBuilder` (and its subclass `ManagerPromptBuilder`) are defined alongside all prompt item types: `RoleDefinition`, `FormatInstruction`, `StrictFormatInstruction`, `ToolInstruction`, `WorkerInstruction`, `EnhancedWorkerInstruction`, `Example`, `DelegationExample`, `History`, and `AgentCapability`. The prompt builder assembles reusable components into a system prompt, while the planner decides how that prompt is combined with history and the current user turn.

- **modules/agent/simple_agent.py**: The SimpleAgent is the conductor of the cognitive orchestra. Its arun method contains the main ReAct loop. It retrieves history from **Memory**, passes it to the **Planner** to get a decision, hands the resulting Action to the **ToolExecutor**, and then formats the tool's output as an "Observation" to be stored back in **Memory** for the next loop. This class is the glue that binds the agent's "anatomy" together.

- **modules/agent/multi_agent_runner.py**: This module showcases a more advanced cognitive architecture. The HierarchicalAgentRunner acts as a meta-conductor for a team of agents. It uses a ManagerPlanner, a specialized planner whose prompt is engineered *not* to use tools directly, but to delegate tasks to worker agents. This demonstrates the power of the framework's modularity: by simply swapping the planner and providing a list of workers, you can elevate a SimpleAgent to a ManagerAgent.

5.4 The Agent's Body: Perception, Action, and Memory

These modules represent how the agent perceives, acts upon, and remembers its environment.

- **The Senses (modules/perception/)**: This is the agent's first point of contact with the world. Components like TextParser implement the AbstractPerception interface to clean, normalize, and prepare raw data before it's passed to the brain.
- **The Hands (modules/action/)**: This is how the agent gets things done.
  - The ToolExecutor is the component that runs the tools. It takes an Action object from the agent, finds the correct tool in the ToolRegistry, and calls its use() method.
  - The ToolRegistry is a simple but vital dictionary that holds all the tools an agent is equipped with.
  - The tools themselves (SafeCalculatorTool, WebSearcherTool, KnowledgeBaseQueryTool, etc.) are the individual "appendages." Each is a self-contained class that inherits from AbstractTool and performs one specific function.
- **The Memory (modules/memory/)**: An agent's power is tied to its memory.
  - WorkingMemory provides short-term context, holding the list of Message objects for the current conversation.
  - The RAG pipeline provides long-term, searchable knowledge. In the current codebase the main collaboration chain is:
    1. 1. `DocumentProcessor` extracts and chunks source files into `Document` objects.
    2. 2. `SentenceTransformerEmbedder` turns those chunks into embeddings.
    3. 3. `ChromaDBVectorStore` or `FaissVectorStore` stores the vectors and source payloads.
    4. 4. `SimpleRetriever` or `CrossEncoderRerankingRetriever` retrieves relevant context, and `KnowledgeBaseQueryTool` exposes that capability to the agent.
  - SummarizingMemory is an advanced component that uses an LLM to automatically compress long conversation histories, elegantly solving the problem of a limited context window.

5.5 The Gateway to the World: The Model Abstraction Layer (modules/mal/)

The MAL is the key to the framework's "Agnostic" principle. Each adapter is a concrete implementation of the `AbstractChatModel` interface. Its job is to act as a translator, converting the framework's internal, standardized `Message` objects into the specific API format required by each LLM provider, and then translating the provider's response back into a `Message` object. The benefit is immense: your agent's logic remains completely decoupled from the specific LLM it is using.

The current adapters are:

- **`OpenAIAdapter`** — connects to OpenAI's chat completion API (e.g., GPT-4o).
- **`AnthropicAdapter`** — connects to Anthropic's Claude models.
- **`HuggingFaceAdapter`** — runs models locally via the Transformers library. Supports both v4 and v5 code paths, including SDPA attention and async streaming on v5.
- **`OllamaAdapter`** — connects to a locally running Ollama server for locally hosted open models.
- **`LoadBalancerAdapter`** — connects to a `vllm_load_manager` service that distributes inference requests across one or more GPU nodes. This adapter is designed for shared lab or classroom infrastructure where multiple agents or users need to share a centralized inference backend. It validates the model against the manager's supported-models endpoint at initialization and retries once on transient failures.

All adapters expose the same `invoke` / `ainvoke` / `stream` / `astream` interface, so swapping one for another requires no changes to the planner, agent, or tool code above it.

5.6 The MCP Integration Layer (modules/mcp/)

The `modules/mcp/` package gives FAIR agents first-class access to tools provided by external Model Context Protocol (MCP) servers. This is more than a demo convenience: the entire MCP client and server infrastructure lives inside the `fairlib` package and is importable directly from `fairlib`.

- **`modules/mcp/client/mcp_client.py`** — **`MCPClient`:** The low-level async client that manages the lifecycle of a connection to an MCP server (either `stdio` subprocess or `SSE` HTTP transport). It handles session initialization, capability negotiation, and tool invocation.

- **`modules/mcp/client/mcp_tool_adapter.py`** — **`MCPToolAdapter`:** Wraps an individual MCP tool so it satisfies the `AbstractTool` interface. This is the bridge that allows an MCP-provided tool to be registered in a standard `ToolRegistry` or a `CompositeToolRegistry`.
- **`modules/mcp/client/mcp_tool_registry.py`** — **`MCPToolRegistry`:** A full `AbstractToolRegistry` implementation that discovers and wraps all tools exposed by one or more MCP servers. Supports an optional `tool_prefix` to avoid name collisions when combining local and MCP tools.
- **`modules/mcp/server/mcp_server.py`** — **`MCPServer`:** Allows a FAIR agent or tool to be exposed as an MCP server, making it callable by other MCP clients such as Claude Desktop.
- **`CompositeToolRegistry`** (in `modules/action/tools/composite_registry.py`): Merges multiple `AbstractToolRegistry` instances into a single interface for the `ToolExecutor`. This is the standard way to combine a local `ToolRegistry` with an `MCPToolRegistry` so the planner can address both transparently.
- **`create_mcp_enhanced_registry`** (in `modules/mcp/__init__.py`): A convenience helper that takes a list of `MCPServerConfig` objects and a local registry, connects to the specified servers, and returns a ready-to-use `CompositeToolRegistry`.

MCP connections are configured with `MCPServerConfig` Pydantic objects (importable from `fairlib`). Each config specifies the transport (`stdio` or `sse`), the command or URL, and any environment variables the server needs. A global `MCPSettings` section in `settings.yml` can hold a list of server configs to connect at startup.

5.7 The Developer's Toolkit: the `fairlib` Import Surface and the Demos

- The primary user-facing API is the package import surface exposed through `fairlib/__init__.py`. It uses lazy loading to make common components discoverable from `from fairlib import ...`, while the deeper module layout remains available for direct imports when a developer wants tighter control or needs a more specific component.
- The `demo_*.py` scripts are executable documentation. They show the intended usage patterns for calculator agents, RAG agents, manager-worker systems, autograder-style committees, model comparison, and structured extraction. Read them as executable documentation: they show how the framework is assembled in practice and are often the fastest route from abstract understanding to working intuition.

Chapter 6

The Prompt Engineering Workbench - Advanced Techniques

Prompts are the source code for agent behavior. While the underlying Large Language Model (LLM) provides the raw intelligence, it is the quality, structure, and precision of the prompt that channels that intelligence into reliable, predictable, and effective action. In the FAIR framework, prompt engineering is not an afterthought; it is a core architectural discipline.

This chapter moves beyond the basics of prompt design. We will treat the prompt system as a *workbench*, dissecting the tools it provides for crafting sophisticated instructions. You will learn not only how to use the existing components but also how to extend the framework with your own custom prompt logic. We will cover:

- **Assembling Complex Prompts with PromptBuilder**: Mastering the pattern of combining static, reusable prompt components with dynamic, task-specific data.
- **Extending the Framework**: A guide to creating your own custom PromptItem types to encapsulate domain-specific instructions.
- **Dynamic Prompts**: Techniques for creating flexible prompt templates that separate static structure from runtime data.
- **Context Reinforcement**: An advanced strategy for combating "instruction drift" and keeping the agent on track during long and complex tasks.

6.1 Assembling Complex Prompts with PromptBuilder

A key challenge in building agentic systems is managing prompt complexity. A simple f-string that works for a single-turn request quickly becomes unmanageable and brittle when dealing with multi-step reasoning, tool descriptions, and strict formatting

constraints. The FAIR framework addresses this with the PromptBuilder class, which encourages a modular and hybrid approach to prompt construction.

The core pattern, demonstrated effectively in the autograder demos (demo_committee_of_agents_essay_ is to use the PromptBuilder to assemble the static, reusable parts of the prompt while using simple f-strings at the very last moment to inject dynamic, task-specific data.

Consider the RubricAligner agent in the essay grading demo. Its role is always the same: to fill out a JSON grade. However, the content it analyzes—the essay, the rubric, and feedback from other agents—changes with every run.

The implementation looks like this:

1. **Static Prompt Template**: A detailed f-string template is defined, containing the agent's instructions and placeholders for dynamic data.

2. **Dynamic Data Injection**: Before calling the LLM, the application populates this template with the specific rubric, content_feedback, style_feedback, and essay for the current grading task.

This hybrid approach provides the best of both worlds: the PromptBuilder can manage the complex, static instructions about the agent's role and output format, while the final f-string injection provides a clear and simple way to insert the data that changes with each execution.

6.2 Extending the Framework - Creating Custom PromptItem Types

The PromptBuilder is designed to be extensible. You are not limited to the default PromptItem types (RoleDefinition, ToolInstruction, etc.). You can create your own domain-specific prompt components by inheriting from the PromptItem abstract base class in core/prompts.py and implementing a render() method that returns a string.

This allows you to encapsulate reusable chunks of your prompt logic into clean, self-contained classes.

6.2.1 Example: Creating a Constraints Item

Imagine you are building an agent that must adhere to a specific set of operational constraints (e.g., "Do not use personally identifiable information," "All financial calculations must be double-checked"). You can create a custom PromptItem to render these constraints consistently.

```python
# In a new file, e.g., 'custom_prompts.py'
from core.prompts import PromptItem
from typing import List

class Constraints(PromptItem):
    """A custom prompt item to render a list of constraints."""
    def __init__(self, constraints_list: List[str]):
        self.constraints = constraints_list
    def render(self) -> str:
        if not self.constraints:
            return ""
        rendered_constraints = "\n".join(f"- {c}" for c in self.constraints)
        return f"# --- OPERATIONAL CONSTRAINTS ---\n{rendered_constraints}"

# In your agent setup code
from custom_prompts import Constraints
# ... create planner ...
my_constraints = Constraints([
    "Do not access external websites unless explicitly told.",
    "All outputs must be formatted in Markdown.",
])

# Add your custom component to the builder's list
planner.prompt_builder.format_instructions.append(my_constraints)
```

Now, every time this planner builds a prompt, it will automatically include the formatted list of operational constraints, ensuring consistency and making your main application code cleaner.

6.3 Dynamic Prompts - Delayed Formatting and Value Injection

For maximum flexibility, you can design a custom PromptItem that acts as a template with placeholders, allowing you to separate the static structure of your prompt from the dynamic data that fills it at runtime.

This pattern is useful when parts of your prompt depend on application state that is only known just before the planner is called.

6.3.1 Example: A Dynamic GoalDefinition Item

Let's create a PromptItem that can be updated with a specific goal and context at runtime.

```python
# In 'custom_prompts.py'
from core.prompts import PromptItem

class DynamicGoal(PromptItem):
    """A prompt item with placeholders for runtime data."""
    def __init__(self, template: str):
        # The template string, e.g., "Your goal is to {goal}. The user's ID is {user_id}."
        self.template = template
        self.values = {}
    def set_values(self, **kwargs):
        """Injects data into the item before rendering."""
        self.values.update(kwargs)
    def render(self) -> str:
        try:
            return self.template.format(**self.values)
        except KeyError as e:
            # Handle cases where a placeholder was not provided
            return f"Error: Missing placeholder value in prompt: {e}"

# In your agent setup code
from custom_prompts import DynamicGoal
# ... create planner ...
dynamic_goal_item = DynamicGoal("Primary objective: {goal}. Context: {context}.")
# Add the dynamic item to the prompt builder
planner.prompt_builder.role_definition = dynamic_goal_item
# --- In your main application loop, just before calling the agent ---
# Set the dynamic values for this specific run
dynamic_goal_item.set_values(
    goal="Summarize the attached document",
    context="The user is a financial analyst."
)
# Now, when you call agent.arun(), the prompt will be correctly populated.
# response = await agent.arun(user_input)
```

This technique gives you the power of structured prompt building while retaining the ability to inject highly dynamic, request-specific information at the last possible

moment.

6.4 Context Reinforcement: Keeping the Agent on Track

A common failure mode in agents that perform long, multi-step tasks is "instruction drift" or "context drift." After many turns of reasoning and observing tool outputs, the LLM can effectively "forget" the initial, high-level instructions from the original system prompt.

To combat this, you can implement a strategy of ***context reinforcement***, where you strategically re-inject key instructions or constraints into the conversation history. This acts as a "nudge" to keep the LLM focused on the most important rules.

6.4.1 Blueprint for a Reinforcement-Aware Agent:

1. **Create a ReinforcementPrompt Component**: This would be a simple data class that holds the Message to be injected and a frequency (e.g., inject_every_n_turns = 4).
2. **Build a ReinforcementAwareAgent**: This custom agent class would inherit from SimpleAgent but override the arun method.
3. **Modify the arun Loop**: Inside the new arun method, before calling the planner, the agent would check the current turn number against the ReinforcementPrompt's frequency. If it's a reinforcement turn, it would manually insert the reminder Message into the history list that gets passed to the planner.

6.4.2 Example ReinforcementAwareAgent Logic Snippet:

```
# Inside the new arun method's loop
if self.reinforcement_prompt and (step %
        self.reinforcement_prompt.frequency == 0) and step > 0:
    # Get the history, inject the reminder, and pass the modified history to the planner
    history_with_reminder = self.memory.get_history()
    history_with_reminder.append(self.reinforcement_prompt.message)
    plan_result = await self.planner.aplan(history_with_reminder,
                                           current_request)
else:
```

```
    # Normal planning call
    plan_result = await self.planner.aplan(self.memory.
    get_history(),
                                           current_request)
```

This technique ensures that critical constraints—such as "You must output valid JSON" or "You must use the final_answer tool when finished"—remain top-of-mind for the LLM, dramatically improving the reliability of agents engaged in long-horizon tasks.

6.5 Specialized Prompt Classes for Smaller Models and Manager Agents

The classes covered so far — `RoleDefinition`, `FormatInstruction`, `Example`, and `ToolInstruction` — are the everyday vocabulary of prompt construction and work well with capable instruction-following models. As soon as you move to smaller local models (3–7B parameters) or build manager agents that must delegate reliably, you will find that generic instructions leave too much room for formatting drift. FAIR provides three additional classes in `core/prompts.py` designed specifically for these situations.

6.5.1 `StrictFormatInstruction`

`StrictFormatInstruction` is a subclass of `FormatInstruction` that adds static factory methods for generating highly explicit DO/DON'T rules. Smaller models often fail silently by wrapping JSON in code fences, nesting objects incorrectly, or emitting multiple Thought/Action pairs in one response. `StrictFormatInstruction` counteracts this with unambiguous positive and negative examples baked directly into the instruction text.

Three factory methods cover the most common scenarios:

```
from fairlib import PromptBuilder, StrictFormatInstruction

builder = PromptBuilder()

# Rules that prevent code fences, nested JSON, and multi-turn
    output
builder.format_instructions.append(
    StrictFormatInstruction.json_output_rules()
)

```

```
# Rules that enumerate valid worker names and forbid using worker tools directly
builder.format_instructions.append(
    StrictFormatInstruction.delegation_rules(["Researcher", "Analyst", "Writer"])
)

# A concrete correct-vs-wrong example rendered inline in the prompt
builder.format_instructions.append(
    StrictFormatInstruction.correct_format_example()
)
```

These three methods are additive: apply all three when prompting a small manager model, or just `json_output_rules` for a worker that only needs to avoid code fences.

6.5.2 `DelegationExample`

Standard `Example` objects work well for multi-turn demonstrations, but manager agents benefit from single-turn examples that show one decision at a time. `DelegationExample` generates exactly that format: a context line (either a user request or a tool observation), a Thought, and a single flat JSON Action, all in one block.

```
from fairlib import PromptBuilder, DelegationExample

builder = PromptBuilder()

# First step example -- triggered by the user request
builder.examples.append(
    DelegationExample.initial_delegation(
        user_request="Find the price of gold and calculate the cost of 10 oz.",
        thought="I need to find the current gold price first. I will delegate this to the Researcher.",
        worker_name="Researcher",
        task="Find the current price of one troy ounce of gold in USD."
    )
)

# Follow-up step -- triggered by an observation
builder.examples.append(
    DelegationExample.followup_delegation(
        observation="The current price of gold is $2,300 per ounce.",
```

```
        thought="I have the price. Now I need to calculate 2300 * 10. I will delegate to the Analyst.",
        worker_name="Analyst",
        task="Calculate 2300 * 10."
    )
)

# Final answer example
builder.examples.append(
    DelegationExample.final_answer_example(
        observation="The result of 2300 * 10 is 23000.",
        thought="I have all the information I need. I can now provide the final answer.",
        answer="Based on a current gold price of $2,300 per ounce, 10 ounces of gold would cost $23,000."
    )
)
```

Using single-turn examples instead of multi-turn narratives has a measurable effect on smaller models: it narrows the pattern they need to match at each step and makes it harder for them to hallucinate a second Thought/Action pair within the same response.

6.5.3 `ManagerPromptBuilder`

`ManagerPromptBuilder` is a `PromptBuilder` subclass that assembles all of the above into a single, opinionated builder designed for manager agents. When `strict_mode=True` (the default), calling `build_system_prompt_string()` automatically appends `StrictFormatInstruction.json_output_rules()`, `StrictFormatInstruction.correct_format_example()`, and `StrictFormatInstruction.delegation_rules(workflow)` if a workflow has been set. This means you only need to provide the role, the worker list, and the examples.

```
from fairlib import ManagerPromptBuilder, DelegationExample

builder = ManagerPromptBuilder(strict_mode=True)

builder.set_role(
    "You are a research team manager. Delegate each sub-task to the right worker."
)
builder.set_workflow(["Researcher", "Analyst", "Writer"])

builder.add_delegation_example(
    context="User Request: Summarize last quarter's AI funding news.",
```

```
    thought="I need current news first. I will delegate to the Researcher.",
    worker_name="Researcher",
    task="Search for AI startup funding news from the last quarter."
)

builder.add_final_answer_example(
    observation="The Writer has produced a two-paragraph summary.",
    thought="All steps are complete. I will present the final summary.",
    answer="Here is a summary of last quarter's AI funding landscape: ..."
)

# Pass builder to ManagerPlanner
from fairlib import ManagerPlanner
planner = ManagerPlanner(llm, workers, prompt_builder=builder)
```

`ManagerPromptBuilder` is particularly valuable when targeting a smaller local model as the manager in a multi-agent system. Larger models such as GPT-4o or Claude typically produce correct delegation format without these scaffolds, but 7B-class models benefit substantially from the added structure.

Chapter 7

The Architect's Room - Advanced Planning and Orchestration

By this point in the guide, you have seen how effective the default ReAct loop can be. It is one of FAIR's strengths because it is understandable, adaptable, and surprisingly capable for many day-to-day agent tasks. Still, good engineering means knowing where a design begins to strain.

This chapter is about that moment. It is about recognizing when step-by-step, reactive reasoning is sufficient and when a task calls for something more deliberate. Many beginners first encounter planning architectures as if they are simply "more advanced" and therefore automatically better. That is not the right lesson. The right lesson is architectural fit. A planning strategy should match the shape of the task.

What follows is therefore not a rejection of ReAct. It is a more mature view of it. We will examine where ReAct shines, where it becomes inefficient or brittle, and how a more explicit plan-and-execute style can improve performance on longer, more dependent workflows.

7.1 A Critical Look at the ReAct Planner

The ReActPlanner and its simpler counterpart, the SimpleReActPlanner, are the workhorses of the FAIR framework. Their logic, as we've seen, is to decide on the single best *next* step. This iterative, reactive approach is powerful for its simplicity and its ability to adapt to new information after every single action. However, for tasks that require foresight and have many dependencies, this model reveals several inherent limitations:

- **Myopia (Short-Sightedness):** The ReAct planner is fundamentally "myopic"—it only sees one step ahead. It generates a single (Thought, Action) pair at a time. For a task like "Research the top three competitors for product X, analyze their market share, and write a summary report," a reactive agent might successfully research the first competitor but has no initial awareness of the

subsequent analysis or writing steps. It can get lost in the details of one sub-task without a clear vision of the full path to completion.

- **Inefficiency and High Cost:** The reactive cycle requires a full LLM call for *every single step*. To find and analyze three competitors, a ReAct agent might make six or more expensive LLM calls (one to decide to search, one to decide to analyze, and so on for each competitor). This not only increases latency and operational costs but is also computationally wasteful for tasks where the overall sequence of actions could have been determined upfront.
- **Instruction Drift and Derailment:** In very long action chains, ReAct agents can suffer from "instruction drift." After ten or fifteen steps, the original user request is buried deep in the conversation history. The agent's attention may be captured by the most recent tool observation, causing it to lose sight of the overarching goal and wander off-task. It can become difficult to maintain a consistent strategy over a long and complex workflow.

While ReAct is a great tool for many jobs, these limitations make it less suitable for complex, long-horizon tasks where a complete strategy can and should be formulated before taking the first step.

7.2 The Plan-and-Execute Paradigm

The Plan-and-Execute model offers a solution to the limitations of ReAct. It fundamentally changes the agent's workflow by separating the cognitive load of *planning* from the mechanical process of *execution*.

The paradigm works in two distinct phases:

1. **PLANNING PHASE:** When the agent receives a complex request, it makes a single, high-level call to a specialized PlanGenerator. The goal of this call is not to decide the *next* action, but to produce a **complete, structured, multi-step plan** that outlines every action required to fulfill the user's request from start to finish. This plan is a static data object, like a detailed project plan or a recipe.
2. **EXECUTION PHASE:** Once the plan is generated and validated, a separate PlanExecutor component takes over. The executor is a simple, deterministic loop. It reads the plan and iterates through each step, calling the necessary tools with the specified inputs. Crucially, the executor does not need to call the LLM during this phase (unless a tool itself calls an LLM). It simply follows the instructions laid out in the plan.

This separation provides enormous benefits for complex tasks: it is far more efficient (one LLM call for planning, many cheap tool calls for execution), it is more reliable (the full strategy is defined upfront), and it is easier to debug (you can inspect the entire plan before a single action is taken).

7.3 Blueprint for a Plan-and-Execute System in FAIR

Integrating this powerful paradigm into the FAIR framework is a matter of architecting new components that still honor the framework's core interfaces. Here is a detailed blueprint for building your own Plan-and-Execute system.

7.3.1 Step 1: Defining the Plan Schema with Pydantic

Before we can generate a plan, we need to define what a "plan" is. A structured schema is non-negotiable for reliability. Using Pydantic, just as we did for FinalGrade in the autograder demos, we can create a canonical data structure for our plans. This ensures that the output from our planner is always valid and predictable.

We'll define two models in a file like core/types.py or a new core/plan_types.py:

```python
# In a file like core/plan_types.py
from pydantic import BaseModel, Field
from typing import List, Dict, Any, Optional

class PlanStep(BaseModel):
    """Represents a single, executable step in a larger plan."""
    step_id: int = Field(..., description="A unique sequential identifier for this step, starting from 1.")
    description: str = Field(..., description="A natural language description of what this step accomplishes.")
    tool_name: str = Field(..., description="The name of the tool to be executed for this step.")
    tool_input: Dict[str, Any] = Field(..., description="The arguments for the tool, provided as a dictionary.")
    dependencies: List[int] = Field(default_factory=list, description="A list of step_ids that must be completed before this step can run.")
    output_variable: Optional[str] = Field(None, description="The variable name to store the output of this step, for use in later steps.")

class ExecutionPlan(BaseModel):
    """Represents a complete, multi-step plan to achieve a user's goal."""
    goal: str = Field(..., description="The original, high-level goal this plan is designed to achieve.")
    steps: List[PlanStep] = Field(..., description="The ordered list of steps to be executed.")
```

This schema gives us a rich, expressive way to define complex workflows, including dependencies between steps.

7.3.2 Step 2: Engineering the Plan-Generation Prompt

With a clear schema, we can now engineer a prompt for a new planner. This prompt must instruct the LLM to act as a meticulous project planner and to output a JSON object that conforms *exactly* to our ExecutionPlan schema. This is a high-stakes prompt that requires clarity, strong format instructions, and a good example.

Here's what the core of a plan-generation prompt might look like:

```
# --- ROLE AND GOAL ---

You are an expert project planner. Your job is to analyze a user's request
and decompose it into a complete, step-by-step plan. Your output MUST be a
single, valid JSON object that conforms to the provided ExecutionPlan schema.

# --- AVAILABLE TOOLS ---
- web_searcher: Searches the web for information. Input: {"query": "..."}
- safe_calculator: Evaluates mathematical expressions. Input: {"expression": "..."}
- report_writer: Writes a summary report from text. Input: {"text_to_summarize": "..."}
# (Additional tools would be listed here)

# --- OUTPUT SCHEMA (MANDATORY) ---

Your entire response must be a JSON object matching this Pydantic schema:

{
    "goal": "The user's original request.",
    "steps": [
        {
            "step_id": 1,
            "description": "First logical step.",
            "tool_name": "...",
            "tool_input": {...},
            "dependencies": [],
            "output_variable": "result_of_step_1"
        },
        {
```

```
            "step_id": 2,
            "description": "Second step, uses result of the first.",
            "tool_name": "...",
            "tool_input": {
                "some_argument": "Use output from step 1: @result_of_step_1"
            },
            "dependencies": [1],
            "output_variable": "..."
        }
    ]
}

# --- EXAMPLE WORKFLOW ---

User Request: "Research the market cap of Apple and Microsoft, and report which is larger."

{
    "goal": "Research the market cap of Apple and Microsoft, and report which is larger.",
    "steps": [
        {
            "step_id": 1,
            "description": "Find the current market cap of Apple Inc.",
            "tool_name": "web_searcher",
            "tool_input": {"query": "current market cap of Apple Inc."},
            "dependencies": [],
            "output_variable": "apple_market_cap"
        },
        {
            "step_id": 2,
            "description": "Find the current market cap of Microsoft Corporation.",
            "tool_name": "web_searcher",
            "tool_input": {"query": "current market cap of Microsoft Corporation."},
            "dependencies": [],
            "output_variable": "microsoft_market_cap"
        },
        {
            "step_id": 3,
            "description": "Compare the two market caps and write a final report.",
```

```
            "tool_name": "report_writer",
            "tool_input": {
                "text_to_summarize": "@apple_market_cap vs @microsoft_market_cap."
            },
            "dependencies": [1, 2],
            "output_variable": "final_report"
        }
    ]
}
```

Notice the @output_variable syntax. This is a convention we've invented to allow the plan to reference the outputs of previous steps, making it dynamic.

7.3.3 Step 3: Creating the PlanGenerator

Now we create a new planner class, PlanGenerator, that implements the AbstractPlanner interface. Its aplan method, however, will not return a (Thought, Action) tuple. Instead, its job is to call the LLM with the plan-generation prompt and return a validated ExecutionPlan object.

```python
# In a new file like modules/planning/plan_and_execute_planner.py
from core.interfaces.planner import AbstractPlanner
from core.interfaces.llm import AbstractChatModel
from core.message import Message
from core.types import ExecutionPlan  # Assuming we defined this
import json

class PlanGenerator(AbstractPlanner):
    def __init__(self, llm: AbstractChatModel, prompt_template: str):
        self.llm = llm
        self.prompt_template = prompt_template

    async def aplan(self, user_request: str, **kwargs) -> ExecutionPlan:
        """Generates a structured, multi-step plan."""
        # 1. Inject the schema and user request into the prompt
        prompt = self.prompt_template.format(
            schema=ExecutionPlan.model_json_schema(),
            user_request=user_request
        )
        # 2. Call the LLM
```

```
        response_message = await self.llm.ainvoke([Message(role="user", content=prompt)])
        response_text = response_message.content
        # 3. Validate and return (self-correction loop)
        try:
            if "```json" in response_text:
                response_text = response_text.split("```json")[1].split("```")[0]
            plan = ExecutionPlan.model_validate_json(response_text)
            return plan
        except (json.JSONDecodeError, ValidationError) as e:
            print(f"Plan validation failed: {e}")
            raise  # Or handle with a retry

# The synchronous `plan` method would just wrap `aplan`.
```

7.3.4 Step 4: Building the PlanExecutorAgent

Finally, we need a new kind of agent whose job is not to think, but to do. The PlanExecutorAgent takes a pre-generated ExecutionPlan and executes it step-by-step.

Its arun method will look very different from SimpleAgent's:

1. It will first call the PlanGenerator to get the complete plan.
2. It will then enter a loop that iterates through the steps of the plan.
3. Inside the loop, it will not call a planner. It will:
    - Resolve dependencies, substituting placeholders like @result_of_step_1 with the actual outputs from previous steps.
    - Call the ToolExecutor with the tool name and inputs for the current step.
    - Store the output of the current step in a dictionary, keyed by its output_variable.
4. After the loop completes, it will return the output of the final step as the result.

```
# In a new file like modules/agent/plan_executor_agent.py
from core.base_agent import BaseAgent
from core.interfaces.executor import AbstractToolExecutor
# ... other imports

class PlanExecutorAgent(BaseAgent):
```

```
    def __init__(self, planner: PlanGenerator, executor: AbstractToolExecutor):
        self.planner = planner
        self.executor = executor

    async def arun(self, user_input: str) -> str:
        # Phase 1: Planning
        print("--- Generating Execution Plan ---")
        try:
            plan = await self.planner.aplan(user_input)
            print(f"OK Plan generated with {len(plan.steps)} steps.")
        except Exception as e:
            return f"Failed to generate a valid plan: {e}"

        # Phase 2: Execution
        print("--- Executing Plan ---")
        step_outputs = {}  # To store results like {"apple_market_cap": "..."}
        for step in sorted(plan.steps, key=lambda s: s.step_id):
            print(f"Executing Step {step.step_id}: {step.description}")
            # Resolve dependencies from previous step outputs
            tool_input = self._resolve_dependencies(step.tool_input, step_outputs)
            try:
                # Execute the tool
                result = self.executor.execute(step.tool_name, tool_input)
                # Store the output if an output_variable is specified
                if step.output_variable:
                    step_outputs[step.output_variable] = result
                    print(f" -> Success. Output stored in: {step.output_variable}")
            except Exception as e:
                return f"Execution failed at step {step.step_id}. Error: {e}"

        # Return the output of the final step, or a summary
        final_result_key = plan.steps[-1].output_variable
        return step_outputs.get(final_result_key,
            "Plan executed successfully, but no final output was specified.")

```

```
    def _resolve_dependencies(self, tool_input: dict, outputs: dict) -> dict:
        # This helper method would parse the tool_input dictionary and
        # replace any "@variable" placeholders with the actual values
        # from the 'outputs' dictionary.
        # (Implementation details omitted for brevity)
        resolved_input = {}
        for key, value in tool_input.items():
            if isinstance(value, str) and value.startswith('@'):
                var_name = value[1:]
                resolved_input[key] = outputs.get(var_name, f"Error: Variable '{var_name}' not found.")
            else:
                resolved_input[key] = value
        return resolved_input
```

Architecting this Plan-and-Execute system will enable you to equip your framework with a powerful new capability. It allows you to build agents that are not only reactive but also strategic, capable of tackling complex, long-running tasks with a level of efficiency, reliability, and foresight that the simple ReAct model cannot match.

Chapter 8

The Crucible - Pre-Execution Plan Validation

Planning ahead is already a major improvement over acting one step at a time without broader foresight. But even a complete plan can still be weak. It can be wasteful, logically inconsistent, incomplete, or unsafe. In high-value or high- consequence systems, that matters a great deal.

This chapter introduces a design habit that becomes more important as systems get more capable: do not assume that the first plausible plan is the right one. Instead, create room in the architecture for review, critique, and refinement before execution begins. In human teams, that is normal. In agentic systems, it should be normal too.

The core idea here is simple and powerful. One agent proposes. Another agent scrutinizes. The system becomes more deliberate not because either agent is perfect, but because the architecture gives mistakes more chances to be caught before they become actions.

8.1 The Philosophy of Adversarial Review

The core idea behind this pattern is simple and battle-tested in human endeavors: a second pair of eyes catches mistakes. In software engineering, this is the mandatory code review. In aviation, it's the pilot's pre-flight checklist. In agentic AI, it's having one LLM-powered agent act as a skeptical "Critic" to find flaws in the plan generated by another LLM-powered "Planner."

This adversarial process forces the system to confront potential issues before they become real problems. It's a proactive quality assurance step that transforms the agent from a purely optimistic executor into a more cautious and deliberate problem-solver. This is not about a lack of trust in the initial planner; it's about building a resilient system that acknowledges the probabilistic nature of LLMs and institutes a process to mitigate the associated risks.

8.2 The Validation Committee - Roles and Responsibilities

This advanced architecture is best implemented as a small, focused committee of agents, each with a clearly defined responsibility. This is a powerful extension of the multi-agent concepts introduced in Chapter 3, orchestrated by a new runner designed specifically for this validation workflow.

- **ChiefPlannerAgent**: This agent is the "creator." It is a SimpleAgent equipped with the PlanGenerator tool designed in Chapter 7. Its sole responsibility is to take a high-level user goal and generate a structured, multi-step ExecutionPlan in JSON format. It is engineered to be creative and to find *a* path to the solution.
- **CriticAgent**: This agent is the "gatekeeper." It is a SimpleAgent whose prompt is engineered to be pessimistic, meticulous, and adversarial. It receives the draft ExecutionPlan from the ChiefPlannerAgent, and its only job is to find flaws. It does not propose solutions; it only identifies problems. Its output is a structured critique, also in JSON, that determines if the plan can proceed.
- **ExecutionManagerAgent**: This agent is the "doer." It is a very simple agent whose logic does not involve any LLM-based planning. It receives an ExecutionPlan *only after* it has been approved by the CriticAgent. Its role is to systematically iterate through the plan's steps, use the standard ToolExecutor to run the specified tools, and ensure the plan is carried out exactly as written.

8.3 Engineering the CriticAgent Prompt

The effectiveness of the entire validation process hinges on the quality of the CriticAgent's prompt. This prompt must be precisely engineered to guide the LLM into a skeptical, analytical mindset. It's not just a set of instructions; it's a "personality" transplant.

The PromptBuilder for the CriticAgent should include the following components:

- **RoleDefinition**: The persona must be explicitly defined as adversarial. For example:

"You are a meticulous and skeptical Quality Assurance agent. Your sole purpose is to find logical flaws, security risks, inefficiencies, and ambiguities in a proposed plan of action. You do not execute the plan or offer solutions; you only find problems. Your critique must be harsh but fair."

- **FormatInstruction**: The output must be a machine-readable JSON object to allow the orchestrator to programmatically check the result. This is non-negotiable.

"# --- MANDATORY RESPONSE FORMAT ---\n" "Your response MUST be ONLY a single JSON object with two keys:\n" "1. is_approved: A boolean (true or false). true only if the plan is logically sound, safe, and complete. false otherwise.\n" "2. critique: A string providing a detailed, step-by-step justification for your decision. If rejecting, you must list every flaw found."

- **Checklist-Driven Reasoning**: The prompt should give the Critic a specific checklist of potential failure modes to look for. This focuses its analysis on the most common types of errors.

"# --- YOUR VALIDATION CHECKLIST ---\n" "1. **Logical Soundness**: Are there any logical gaps? Do the steps flow correctly? Does each step logically follow from the previous ones?\n" "2. **Completeness**: Does the plan fully address all parts of the original goal? Is anything missing?\n" "3. **Efficiency**: Is there a more direct or less resource-intensive way to achieve the goal? Are there redundant or unnecessary steps?\n" "4. **Risk & Safety**: Does the plan involve executing code, accessing files, or calling external APIs? If so, are there proper security checks, like input sanitization? Could any step lead to unintended consequences?\n" "5. **Dependency Management**: If the output of one step is the input for another, is this dependency clearly handled?"

8.4 The ValidatedPlannerRunner - The Orchestrator

The HierarchicalAgentRunner from our previous multi-agent systems is not suitable for this validation loop. We need a new orchestrator, the ValidatedPlannerRunner, to manage the three-agent committee and the plan refinement cycle.

The arun() method of this new runner would implement the following logic:

1. **Receive Goal**: The runner is called with the high-level user request.
2. **Generate Initial Plan**: It calls ChiefPlannerAgent.arun(goal) to generate the first draft of the ExecutionPlan.
3. **Enter Validation Loop**: The runner begins a loop (e.g., for a maximum of 3 refinement attempts).
4. **Submit for Review**: Inside the loop, it calls CriticAgent.arun(plan), passing the current draft plan as the input.

5. **Check Verdict**: The runner parses the CriticAgent's JSON response.
   - If is_approved is true, the plan is considered validated. The runner breaks out of the loop and proceeds to execution.
   - If is_approved is false, the plan is rejected.
6. **Refine the Plan**: On rejection, the runner constructs a new, targeted prompt for the ChiefPlannerAgent. This prompt includes the original goal *and* the specific feedback from the critic. For example:

"Please generate a plan to achieve GOAL. Your previous attempt was rejected for the following reason: [Insert critique text here]. Generate a new, improved plan that specifically addresses all of these issues."

7. **Re-plan**: The runner calls ChiefPlannerAgent.arun() with this new, feedback-informed prompt to get a revised plan, and the loop continues.
8. Execute or Fail:
   - If the loop completes with an approved plan, the runner passes the final, validated ExecutionPlan to the ExecutionManagerAgent for execution.
   - If the loop finishes after exhausting all retries without an approved plan, the runner terminates and returns a failure message to the user, explaining that a reliable plan could not be formulated.

This refinement cycle is a form of automated, structured debate. It uses the LLM's own capabilities to iteratively improve its output, resulting in a final plan that is significantly more robust than one produced in a single pass.

Chapter 9

The Agent's Mind - Architecting Advanced Memory Systems

An agent's power is inextricably tied to its ***memory***. An agent with no memory is a purely reactive machine, answering each query in isolation. An agent with a perfect, infinite memory would be omniscient but computationally impractical. The art of agentic engineering lies in designing a memory architecture that is both powerful and practical, enabling an agent to learn, adapt, and recall information effectively without being overwhelmed by the sheer volume of data it encounters.

This chapter explores some of the memory architectures that transform a basic agent into a knowledgeable and stateful entity. We will dissect the different tiers of memory within the FAIR framework, moving from the simple, ephemeral context of a single conversation to the design of persistent, per-user knowledge bases. You will learn how to give your agents the ability to dynamically choose what to remember and even how to implement strategies for managed forgetting, a crucial capability for preventing knowledge bloat and maintaining relevance over time.

9.1 The Two Tiers of Memory in FAIR

The FAIR framework conceptualizes memory into two primary tiers, each serving a distinct purpose and corresponding to different modules within the codebase.

9.1.1 Short-Term Memory: The Conversational Context

This is the agent's working RAM, the ephemeral scratchpad that holds the immediate context of a single, ongoing conversation. It's what allows an agent to understand pronouns, follow a multi-step instruction, and remember what was said just a few turns ago.

- **WorkingMemory (modules/memory/base.py)**: This is the simplest implementation, a list of Message objects that stores the recent back-and-forth. It's

fast, efficient, and perfect for most single-task agents. However, its fixed size means it can lose context in very long conversations.

- **SummarizingMemory (modules/memory/summarization.py)**: This is a more advanced form of short-term memory designed to combat the context window limitations of LLMs. When a conversation exceeds a certain length, this module uses an LLM to "compress" the middle portion of the history into a concise summary. The memory then consists of the original system prompt, the new summary, and the most recent few messages. This preserves the agent's core instructions and recent context while preventing the prompt from becoming excessively long and costly.

9.1.2 Long-Term Memory: The Knowledge Library (RAG)

This tier represents the agent's persistent knowledge, the library it can consult to answer questions, verify facts, and ground its responses in a source of truth. The FAIR framework implements this through a robust Retrieval-Augmented Generation (RAG) pipeline.

- **The RAG Pipeline**: This is not a single component but a system of interconnected modules working in concert. It begins with an **Embedder** (modules/memory/embedder.py), such as the SentenceTransformerEmbedder, which converts text into numerical vectors. These vectors are stored in a **VectorStore**. FAIR provides two implementations: `FaissVectorStore` (modules/memory/vector_faiss.py) for persistent, production-grade retrieval with optional GPU acceleration and cross-encoder re-ranking, and `ChromaDBVectorStore` (modules/memory/vector_store.py) for simpler in-memory or lightly-persistent setups. When a query is made, a **Retriever** (modules/memory/retriever.py) uses the vector store to find the most semantically similar document chunks. This entire process is exposed to the agent through a dedicated tool, the **KnowledgeBaseQueryTool** (modules/action/tools/knowledge_tool.py), allowing the agent to consciously decide when to access its long-term memory.

9.2 Blueprint for an Advanced Memory Architecture

The default memory systems are a good and practical start but have two key limitations for many real-world applications: they are ***ephemeral*** (lost when the application restarts) and ***global*** (all users interact with the same memory). To build a truly personalized, multi-user application, you must architect a more advanced memory system. Here is a blueprint for doing so.

9.2.1 Scoped and Persistent Memory: Giving Each User Their Own Brain

To create a system where each user has a unique conversational history and knowledge base, you must move from in-memory stores to persistent, uniquely identified ones.

- **Persistent Short-Term Memory**: Instead of the default WorkingMemory, you would implement a custom memory module that saves the conversation history to a persistent, session-based store. A key-value database like Redis is an excellent choice for this, as it provides the speed needed for rapid conversational turns. Each user session would have a unique key (e.g., session_history_:{user_id}), and each new message would be appended to that user's list in Redis.
- **Persistent and Scoped Long-Term Memory**: The RAG pipeline can be made persistent and multi-tenant by making a simple but critical change in how you initialize the vector store. When using ChromaDBVectorStore, instead of using a single, hardcoded collection name, you can create a unique collection for each user or session.

```python
# --- Option A: ChromaDB (simpler, good for development) ---
user_id = "user_12345"
user_specific_collection_name = f"user_knowledge_base_{user_id}"

user_vector_store = ChromaDBVectorStore(
    client=chromadb.PersistentClient(path="/path/to/persistent/db"),
    collection_name=user_specific_collection_name,
    embedder=SentenceTransformerEmbedder()
)

# --- Option B: FAISS (recommended for production workloads) ---
# FaissVectorStore persists to disk and supports GPU acceleration.
# Each user gets their own index directory.
from fairlib.modules.memory.vector_faiss import FaissVectorStore
from fairlib import SentenceTransformerEmbedder

user_index_dir = f"./data/faiss_indexes/{user_id}"
user_vector_store = FaissVectorStore(
    embedder=SentenceTransformerEmbedder(),
    index_dir=user_index_dir
)

```

```
# In both cases, wrap in LongTermMemory for use with the agent
user_long_term_memory = LongTermMemory(user_vector_store)
```

This approach transforms the RAG system from a global knowledge base into a personal one, allowing an agent to be an expert on *that specific user's* documents. For most production deployments, the FAISS path is preferred because it persists the index as files on disk, supports optional GPU acceleration, and integrates with cross-encoder re-ranking through `retriever_rerank.py`.

9.2.2 Dynamic Memory: The MemorizationTool

In a standard ReAct loop, an agent can only read from its long-term memory. It cannot decide to add new information to it based on the conversation. A more advanced agent should be able to learn from its interactions. This can be achieved by creating a MemorizationTool.

- **The Tool's Function**: This custom tool would take a string of text as input (a "fact" or piece of information) and its use method would call the add_documents method of the LongTermMemory's vector store.
- **Updating the Prompt**: The agent's intelligence doesn't come from the tool alone, but from knowing when to use it. You would need to update the Planner's PromptBuilder to teach the agent this new skill. You would add a ToolInstruction for the new MemorizationTool and, critically, add a new Example demonstrating a workflow where the agent uses it.

Example Prompt Snippet

```
# --- Example: Learning a new fact ---

user: My favorite color is blue. Please remember that.

A:
Thought: The user has given me a specific fact to remember for later. I
should use the memorization_tool to commit this to my long-term knowledge base.

Action:
tool_name: memorization_tool
tool_input: The user's favorite color is blue.
```

This gives the agent ***agency over its own memory***, allowing it to dynamically build its knowledge base over time.

9.2.3 Managed Forgetting: Pruning and Time-Based Decay

An infinite memory is a liability. Old, irrelevant information can "pollute" RAG search results, leading to incorrect or outdated answers. A truly advanced memory system must have a strategy for forgetting.

- **Adding Timestamps to Metadata**: The first step is to augment the data you store. When adding documents to your VectorStore, include a timestamp in the metadata dictionary.
- **Implementing Pruning Logic**: With timestamps in place, you can implement several forgetting strategies:
  - **Periodic Pruning**: A background process could run periodically (e.g., once a day) to scan the vector store and delete all documents with a timestamp older than a certain threshold (e.g., 90 days).
  - **Agent-Driven Forgetting**: You could create a ForgettingTool that allows a privileged "admin" agent to manually trigger the pruning of outdated information related to a specific topic.
  - **LRU (Least Recently Used) Cache Logic**: For very large-scale systems, you could build a more sophisticated memory manager that tracks not just the creation timestamp but also the last-accessed time of each document chunk. When the knowledge base reaches a certain size, the system could automatically prune the chunks that haven't been retrieved recently, ensuring that the most relevant information is always the most accessible.

Architecting these advanced memory systems will allow you to move beyond simple chatbots and create agents that have a genuine sense of history, identity, and evolving knowledge—the true hallmarks of an intelligent system.

Chapter 10

The Network - Architecting Agent Communication

Most early FAIR examples are intentionally compact. A single agent handles a query, or a manager delegates work to a few workers inside one process. That is a good place to start because it keeps the control flow visible. But it is not the full story of what agentic systems may eventually need to become.

As soon as agents begin collaborating more seriously, communication becomes an architectural concern of its own. How do agents pass work, results, requests, or status updates? Who mediates those exchanges? Where are messages stored? What happens when participants are not all running in the same process or even on the same machine?

This chapter looks at those questions in a way that is meant to help both newcomers and more advanced builders. We begin with the communication patterns already visible in FAIR today, then use them as a stepping stone toward a more distributed mental model.

10.1 Communication Patterns in the FAIR Framework

The framework supports two primary modes of inter-agent communication, each with its own strengths and limitations.

10.1.1 Implicit Communication: Orchestration-Driven Workflows

This is the default pattern demonstrated in the multi-agent autograder demos and is the simplest to implement and debug.

- **How it Works**: Communication is not direct between agents but is mediated entirely by a central orchestrator, the HierarchicalAgentRunner. A "Manager" agent decides which "Worker" to task, but it doesn't send a message directly to the worker. Instead, it returns a delegate action. The HierarchicalAgentRunner intercepts this action, calls the specified worker's arun method with the task, receives the worker's final response, and then packages that response as an "Observation" to be added to the Manager's memory for its next reasoning cycle.
- **Strengths**: This model is predictable, easy to trace, and debuggable. The flow of control is linear and centralized, making it straightforward to understand the state of the system at any given moment.
- **Limitations**: The orchestrator is a single point of failure and a performance bottleneck. Every interaction must pass through the central runner, which is inefficient for large-scale systems. Most importantly, it limits agent autonomy; workers cannot initiate communication, only respond to delegated tasks.

10.1.2 Explicit Communication: The In-Memory Communicator

The framework anticipates the need for more direct communication through its AbstractCommunicator interface.

- **How it Works**: The InMemoryCommunicator provides a baseline implementation of this interface. It acts as a simple, single-process message bus, using a Python dictionary to hold message queues for each registered agent. An agent can use this communicator to directly send a message to another agent's queue.
- **Strengths**: It allows for direct agent-to-agent messaging, decoupling them from a central orchestrator. This is a step towards greater autonomy.
- **Limitations**: As its name implies, it only works for agents running within the **same Python process**. It is not suitable for distributed systems where agents may be running on different machines or in different containerized environments. It's a proof-of-concept for the interface, not a scalable solution.

10.2 Blueprint for a Distributed Communication Backbone

To enable scalable and truly autonomous multi-agent systems, we must evolve beyond the limitations of the in-memory approach. This requires an external, robust message broker to serve as the communication backbone. This blueprint outlines how to extend the FAIR framework to build such a system, using Redis as an example.

10.2.1 Implement a New AbstractCommunicator

The first step is to create a new class, RedisCommunicator, that implements the AbstractCommunicator interface. Instead of using an in-memory dictionary, this class would connect to a Redis server and leverage its powerful Pub/Sub (Publish/Subscribe) capabilities.

- **asend_message(recipient_agent_id, message)**: This method would not append to a local list but would instead PUBLISH the message to a specific Redis channel, likely named after the recipient (e.g., agent_channel:<recipient_agent_id>). The message object itself would be serialized (e.g., to a JSON string) before being sent.
- **areceive_message(agent_id)**: This method would involve subscribing to the agent's dedicated channel (agent_channel:<agent_id>) and listening for incoming messages. This allows for an event-driven design where agents can react to messages as they arrive.

10.2.2 Create Communication Tools

For agents to use this new communication backbone autonomously, they need tools. As part of their ReAct loop, they must be able to decide to communicate. We would create tools like:

- **SendMessageTool**: An agent could use this tool to send a direct message to another specific agent.
  - **Description**: "Sends a message to a specific agent. Input must be a dictionary with ' recipient_id'and ' message_content'."
  - **Action**: {"tool_name": "send_message", "tool_input": {"recipient_id": "DataAnalyst_Agent_02", "message_content": "Here is the data you requested."}}
- **BroadcastMessageTool**: This tool would allow an agent to send a message to all other agents in the network (or a specific group).
  - **Description**: "Broadcasts a message to all other agents in the system."
  - **Action**: {"tool_name": "broadcast_message", "tool_input": "System-wide alert: Data processing is complete."}

These tools would use the RedisCommunicator instance to publish messages to the appropriate Redis channels.

10.2.3 Build a CommunicatorAwareAgent

This new agent class represents the most significant evolution from the SimpleAgent. It would be designed from the ground up to be event-driven.

- **Main Listen Loop**: Instead of being passively called by an orchestrator, a CommunicatorAwareAgent would have a primary listen loop. This loop would subscribe to its Redis channel and wait for messages.
- **Triggered Execution**: The arrival of a message would trigger the agent's arun method. The content of the message would serve as the user_input for its reasoning cycle.
- **True Autonomy**: This architecture fundamentally changes the paradigm. Agents are no longer just workers waiting for a manager; they are autonomous entities that can listen, react, and proactively communicate with each other. This removes the HierarchicalAgentRunner bottleneck and enables truly decentralized and scalable collaboration.

10.2.4 Standardized Message Format and Agent Discovery

For a distributed system to work, communication must be standardized.

- **Agent Directory Service**: How does an agent know who to talk to? A simple implementation could involve a shared key in Redis where agents register themselves upon startup, listing their agent_id and role_description. This allows other agents to discover available peers and their capabilities.
- **Standardized Message Structure**: Every message passed through the communicator should have a clear, consistent structure, including:
  - **Headers**: Sender ID, Recipient ID (or broadcast flag), Message ID, Timestamp, Message Type (e.g., "request", "response", "alert").
  - **Payload**: The actual content of the message, which could be a simple string or a complex, serialized object.

By building these components—a Redis-backed communicator, communication-enabling tools, and an event-driven agent class—we transform the FAIR framework from a system for building isolated agents into a platform for orchestrating a true "Internet of Agents."

Chapter 11

The Agent's Toolkit: Building and Integrating Tools

Tools are where an agent stops merely talking about work and begins doing work. A planner can reason beautifully, but without tools it remains confined to what the base model already knows and whatever it can express as text. The moment you add tools, the agent can calculate, retrieve, search, extract, grade, graph, read, write, and interact with systems beyond the model itself.

For beginners, tool building is one of the most important topics in the whole guide because it changes how you think about agents. An agent is not just a prompt with a personality. It is a reasoning loop connected to capabilities. The quality of those capabilities, and the clarity with which you expose them to the planner, has a huge effect on how useful the agent becomes.

This chapter explains how FAIR tools work, how to design them well, how to register them, and how MCP-based tools fit into the same overall picture. The goal is not only to show you the interface. The goal is to help you build tools that planners can select reliably and that developers can maintain safely.

11.1 Why Tools Matter

A language model is good at pattern completion and often surprisingly good at reasoning, but there are limits to what it should be trusted to do on its own. Some tasks require exact arithmetic. Some require access to live information. Some require deterministic program logic. Some require reading from systems the model cannot otherwise see. Tools exist to close that gap.

In FAIR, a tool can serve at least four major roles:

- **Precision:** use code to do something the model should not approximate, such as arithmetic or symbolic manipulation.
- **Access:** reach information or services outside the model's frozen knowledge, such as weather, web search, or a document store.

- **Control:** ensure that a complex or sensitive action happens through a bounded piece of code rather than through free-form model behavior.
- **Structure:** turn messy capabilities into named, reusable components that can be shared across agents and demos.

When you are designing an agent, a good question to ask is not only "What should this agent know?" but also "What should this agent be able to *do* through tools?" That shift in thinking often leads to better designs.

11.2 What a FAIR Tool Is

At the local framework level, a FAIR tool is a Python class that follows the contract defined by `AbstractTool` in `fairlib/core/interfaces/tools.py`. That contract is intentionally simple. A tool exposes:

- a `name`, which is the identifier the planner will use;
- a `description`, which tells the planner what the tool is for; and
- a `use(self, tool_input: str) -> str` method, which performs the action and returns the observation as a string.

That simplicity is important. It means tools are easy to reason about and easy to register. It also means the planner can treat many capabilities uniformly. Whether the tool is a calculator, a retriever-backed knowledge query, a grading utility, or a plotting helper, the agent sees the same conceptual interface: choose a tool name, send it input, get back an observation.

11.2.1 The Three Fields Every Beginner Should Care About

1. **The tool name.** This should be stable, descriptive, and machine-friendly. Use snake case and keep it clear enough that both humans and models can infer its purpose.
2. **The description.** This is often the single most important part of the tool. The planner chooses tools largely by matching the user's need against the descriptions available in the prompt. A vague description leads to vague tool usage.
3. **The `use()` implementation.** This is where your deterministic program logic lives. It should validate its input, handle failures gracefully, and return a result the agent can reason over on the next loop.

11.3 The Tool Lifecycle in FAIR

A good way to learn tools is to follow the full lifecycle from idea to use. Most FAIR tools pass through the same stages.

11.3.1 1. Identify the Capability

Start with the problem, not the class. Ask what concrete capability is missing from the agent. Does it need exact math? File access? Retrieval from a local knowledge base? Safe code execution for a lab setting? External search?

A tool should generally have a narrow and understandable purpose. Beginners are sometimes tempted to create one giant "do everything" tool. That usually makes life harder for both the planner and the human developer. Smaller, well-defined tools are easier to describe, easier to test, and easier to reuse.

11.3.2 2. Design the Interface

Before writing code, decide:

- what the tool is called;
- what kinds of requests should cause the planner to choose it;
- what input format it expects; and
- what the returned observation should look like.

That last point matters more than many newcomers expect. The output of a tool is not only for the human reader. It becomes context for the next reasoning step. A good tool observation should therefore be clear, informative, and easy for the planner to build on.

11.3.3 3. Implement the Tool Class

Here is a small example of a local FAIR tool that writes text into a controlled output directory. It is intentionally simple, but it illustrates the main ideas.

```python
import json
from pathlib import Path
from fairlib.core.interfaces.tools import AbstractTool

class FileWriterTool(AbstractTool):
    name = "file_writer"
```

```
    description = (
        "Use this tool to write text content to a file inside the safe output "
        "directory. Provide JSON with keys 'filename' and 'content'. "
        "Use it when the user asks you to save notes, text, reports, or code."
    )

    def __init__(self, base_directory: str = "agent_output"):
        self.base_path = Path(base_directory)
        self.base_path.mkdir(parents=True, exist_ok=True)

    def use(self, tool_input: str) -> str:
        try:
            data = json.loads(tool_input)
            filename = data.get("filename")
            content = data.get("content")

            if not filename or not isinstance(filename, str):
                return "Error: 'filename' must be a non-empty string."
            if content is None:
                return "Error: 'content' is required."

            safe_filename = Path(filename).name
            output_path = self.base_path / safe_filename
            output_path.write_text(str(content), encoding="utf-8")
            return f"Success: wrote content to {output_path}"

        except json.JSONDecodeError:
            return (
                "Error: tool_input must be valid JSON with keys "
                "'filename' and 'content'."
            )
        except Exception as e:
            return f"Error: {e}"
```

11.3.4 4. Register the Tool

A tool is not available to the agent until it has been placed in a `ToolRegistry`. The registry is the catalog of capabilities the planner will see and the executor will use.

```python
from fairlib import (
    ToolRegistry,
    ToolExecutor,
    WorkingMemory,
    SimpleAgent,
    ReActPlanner,
    OpenAIAdapter,
)
from my_project.file_writer_tool import FileWriterTool

llm = OpenAIAdapter("gpt-4")
registry = ToolRegistry()
registry.register_tool(FileWriterTool())

executor = ToolExecutor(registry)
memory = WorkingMemory()
planner = ReActPlanner(llm, registry)

agent = SimpleAgent(
    llm=llm,
    planner=planner,
    tool_executor=executor,
    memory=memory,
)
```

Once the registry is passed into the planner, the tool descriptions become part of the planner's available-capability context. Once the same registry is passed into the executor, the action emitted by the planner can actually be carried out.

11.3.5 5. Let the Agent Use It

If the user asks something like "Save this summary as notes.txt," the planner can match that request against the `file_writer` description, emit an action for that tool, and let the executor run it. On the next loop, the tool's success or error string becomes an observation. The planner then decides whether to continue, repair, or finalize.

This is the pattern to remember: **descriptions drive selection, code performs the work, observations drive the next step.**

11.4 Why the Description Matters So Much

Many tool bugs are really description bugs. A developer may build a perfectly functional tool and then wonder why the planner never chooses it, chooses it at the

wrong time, or feeds it the wrong kind of input. Often the issue is that the description does not teach the model enough about when and how to use the tool.

A good tool description usually answers four questions:

- **What does this tool do?**
- **When should it be used?**
- **What input format does it expect?**
- **What kinds of tasks or questions is it especially good for?**

Compare these two styles:

- **Weak:** "Writes a file."
- **Stronger:** "Use this tool to write notes, reports, code, or text to a file in the safe output directory. Provide JSON with keys 'filename' and 'content'. Use it when the user asks you to save or export generated text."

The second version gives the planner many more anchors: what the tool is for, what language is likely to signal it, and what input structure it expects.

11.5 Complex Inputs: Why JSON Is a Common Pattern

The `use()` contract accepts a single string, but that does not mean your tool must be limited to one trivial argument. In FAIR, the common pattern for multi-field inputs is to serialize them as JSON inside `tool_input`.

This is how several more advanced tools in the framework operate. It is also a good beginner pattern because it scales naturally:

- a calculator may only need a plain expression string;
- a file writer may need `filename` plus `content`;
- a grader may need submission text, rubric information, and additional instructions; and
- a code-execution tool may need source code, language, and execution metadata.

If your tool expects structured input, say so directly in the description and validate it inside the tool.

11.6 Dependency Injection and Reusable Tools

Tools do not have to be stateless. In fact, many of the most useful FAIR tools are configured with dependencies when they are constructed. This is a healthy design pattern because it keeps the tool itself generic while letting the application decide what backend or resource it should use.

A good example is `KnowledgeBaseQueryTool`, which takes a retriever rather than being hardcoded to one storage backend. That means the same tool concept can sit on top of ChromaDB, FAISS, reranked retrieval, or some future retrieval path.

```
from fairlib import ToolRegistry, KnowledgeBaseQueryTool,
    SimpleRetriever

retriever = SimpleRetriever(vector_store)
knowledge_tool = KnowledgeBaseQueryTool(retriever)

registry = ToolRegistry()
registry.register_tool(knowledge_tool)
```

This is worth noticing as a design lesson. A tool often represents a capability, not a single hardwired implementation.

11.7 Built-In Tools as Learning Examples

The FAIR codebase already contains several useful examples that new developers should study:

- **`SafeCalculatorTool`:** a clean example of a narrow, deterministic tool.
- **`AdvancedCalculusTool`:** a more specialized mathematical tool.
- **`WebSearcherTool`:** an example of a tool that reaches outside the model's static knowledge.
- **`WeatherTool`:** a simple example of structured external lookup.
- **`GraphingTool`:** a capability that produces a concrete artifact.
- **`KnowledgeBaseQueryTool`:** a retrieval-backed tool for RAG.
- **`GradeEssayFromRubricTool`** and **`GradeCodeFromRubricTool`:** examples of tools that wrap more involved evaluation logic.
- **`CodeExecutionTool`:** an important example for discussing risk, validation, and sandboxing requirements.

- **`WebDataExtractor`:** a tool for fetching and extracting structured data from web pages, demonstrating how to wrap HTTP-based capabilities.
- **`CompositeToolRegistry`:** not a tool itself, but a registry that merges multiple `AbstractToolRegistry` instances into one. It is the standard pattern for combining local tools with MCP-backed tools so the executor and planner see a single unified toolbelt. See the MCP section below for usage.

Reading these tools is one of the fastest ways to build intuition for your own. Try to notice how their descriptions differ, how much validation they perform, and what kind of observation strings they return.

11.8 MCP Tools: Extending FAIR Beyond Local Classes

FAIR's local tool pattern is only one half of the modern story. MCP support is now a first-class capability built directly into the `fairlib` package. An agent can work with tools provided by external or out-of-process MCP servers using the same planner, executor, and registry interfaces it uses for local tools. The entire MCP client and server infrastructure lives in `fairlib/modules/mcp/` and is importable directly from `fairlib`.

11.8.1 The Big Idea Behind MCP

Model Context Protocol (MCP) provides a standardized way for tools to be exposed by a server and consumed by a client. In practical terms, it means that another service can publish usable tools over a well-defined protocol, and your agent can bring those tools into its workflow without custom ad hoc glue for every new integration.

For students, the most useful way to think about this: MCP turns tools into network-reachable or process-reachable capabilities that still look like ordinary FAIR tools to the planner.

11.8.2 The MCP Module Inside fairlib

All MCP components are importable from `fairlib`:

- **`MCPServerConfig`** — a Pydantic config object that describes how to connect to one MCP server. Set `transport` to `"stdio"` for subprocess servers or `"sse"` for HTTP servers. For stdio servers, provide `command` and `args`; for SSE servers, provide `url`.

- **`MCPToolRegistry`** — an `AbstractToolRegistry` implementation that connects to one or more MCP servers and wraps each discovered tool as an `MCPToolAdapter` (which in turn satisfies `AbstractTool`). Use an optional `tool_prefix` to prevent name collisions with local tools.
- **`CompositeToolRegistry`** — merges any number of `AbstractToolRegistry` instances into one. Hand this to the `ToolExecutor` and the planner sees a unified tool list regardless of whether each tool comes from a local class or a remote MCP server.
- **`create_mcp_enhanced_registry`** — a convenience coroutine in `fairlib.modules.mcp` that accepts a local registry and a list of `MCPServerConfig` objects, connects to each server, and returns a ready-to-use `CompositeToolRegistry`.
- **`MCPServer`** — allows a FAIR agent or tool to be *served* as an MCP server so that other MCP clients (including Claude Desktop) can call it.

`MCPSettings` in `settings.yml` holds a list of server configs to connect at startup, making it easy to wire up MCP servers through configuration rather than code.

11.8.3 Two Integration Patterns

Pattern 1: Helper-driven combined registry. The simplest path uses `create_mcp_enhanced_registry`:

```python
from fairlib import (
    ToolRegistry, SafeCalculatorTool, MCPServerConfig
)
from fairlib.modules.mcp import create_mcp_enhanced_registry

local_registry = ToolRegistry()
local_registry.register_tool(SafeCalculatorTool())

mcp_config = MCPServerConfig(
    name="filesystem",
    transport="stdio",
    command="python",
    args=["demos/mcp/mcp_filesystem_server.py"]
)

# Returns a CompositeToolRegistry containing both local and MCP tools
combined_registry = await create_mcp_enhanced_registry(
    local_registry=local_registry,
    server_configs=[mcp_config]
)
```

```

executor = ToolExecutor(combined_registry)
planner = SimpleReActPlanner(llm, combined_registry)
```

Pattern 2: Explicit registry composition. For tighter control, build the registries separately and combine them manually:

```
from fairlib import (
    MCPServerConfig, CompositeToolRegistry, ToolRegistry
)
from fairlib.modules.mcp.client.mcp_tool_registry import
    MCPToolRegistry

local_registry = ToolRegistry()
local_registry.register_tool(SafeCalculatorTool())

mcp_registry = MCPToolRegistry(tool_prefix="brave")
brave_config = MCPServerConfig(
    name="brave-search",
    transport="sse",
    url="http://localhost:3000/sse"
)
await mcp_registry.add_server(brave_config)

combined = CompositeToolRegistry([local_registry,
    mcp_registry])
executor = ToolExecutor(combined)
```

Both patterns result in a registry the planner and executor use identically to a plain `ToolRegistry`. The MCP complexity is entirely encapsulated behind the `AbstractToolRegistry` interface.

11.8.4 Why This Matters Architecturally

MCP matters because it widens the tool horizon without compromising the agent design. FAIR's planning loop, registry concept, and executor remain unchanged. What changes is where capabilities live. A classroom demo can start with one local tool; the same agent design can later pull in filesystem access, web search from a Brave Search Docker container, or any other MCP-compatible service with minimal code change.

MCP is also a strong example of the interoperability principle: FAIR does not require every capability to be reimplemented in one style. Its job is to let agents reason over capabilities cleanly, whether those capabilities come from local classes, retrieval systems, or external tool servers.

11.9 Best Practices for Tool Builders

- **Keep each tool narrow.** A planner usually performs better with several clear tools than with one sprawling, ambiguous tool.
- **Write descriptions for the planner.** Think about how the model will decide when to use the tool.
- **Validate inputs aggressively.** Tool input is model-generated text. It should be treated as untrusted.
- **Return useful observations.** A good observation helps the next reasoning step succeed.
- **Handle failures as strings, not crashes.** A returned error can become context for repair. A crashed process usually ends the loop.
- **Prefer explicit formats for complex inputs.** JSON is common because it scales cleanly and makes validation easier.
- **Test tools outside the agent first.** If a tool is unreliable by itself, it will be worse once a planner starts using it automatically.

11.10 Security Notes You Should Take Seriously

Tool execution is where an agent starts touching real systems, which means this is where risks become concrete.

- **Filesystem and code tools are sensitive.** Restrict paths, validate arguments, and avoid broad privileges.
- **The current `CodeExecutionTool` should be treated as a lab or demo capability unless it is wrapped in a real sandbox.** If execution matters in a higher-trust environment, use proper isolation such as containers or more hardened sandboxing approaches.
- **Least privilege should be the default.** A tool should only be able to do the one job it was created to do.
- **External tool servers deserve the same caution.** MCP expands capability, but it also expands the surface area of what the agent can reach. Be deliberate about which servers you trust and what they expose.

11.11 Closing Perspective

Learning to build tools changes the kind of systems you can build with FAIR. It is the difference between a model that can speak and a system that can act in a bounded, inspectable, and reusable way. Once you become comfortable designing capabilities as tools, the rest of the framework starts making even more sense. Planners choose among capabilities. Executors carry them out. Observations feed back into reasoning. That cycle is the heart of agentic development, and tools are the bridge that make it useful.

Chapter 12

Installing FAIR-LLM and Working Through the Demo Repository

This chapter shifts from framework internals to day-one practice. Up to this point, this guide has focused on how FAIR is organized, how agents reason, how tools fit into the loop, and how more advanced behaviors like RAG, structured output, and multi-agent coordination are assembled. That architectural understanding matters, but many students and new developers learn fastest when they can install a working package, run something concrete, and then trace the code with purpose.

That is exactly what the public demo repository is for.

A useful aspect of FAIR's current distribution model is that a developer does *not* need direct access to the full core repository in order to start building with the framework. The public package can be installed directly, and the public demo repository provides runnable examples that show how to assemble agents, configure models, attach tools, ground agents in documents, serialize configurations, coordinate agent teams, and even wrap parts of the framework in a browser-based application. In other words, the demos are not just sample files; they are a guided tour of the framework's most important patterns.

This chapter therefore has two jobs. First, it explains how to get a clean, working environment ready for the public FAIR package and the demo repository. Second, it walks through every major demo script and explains what each one is trying to teach. The goal is not simply to tell you *how* to run the demos, but to help you understand *why each one exists*, what architectural lesson it captures, and which demos you should study first depending on what you want to build.

12.1 The Public Package and the Public Demo Repository

For new users, one of the most important things to understand is that the package name, the import name, and the demo repository are related, but they are not the same thing.

- The installable package is distributed as `fair-llm`.
- The Python import surface used in the demos is `fairlib`, as in `from fairlib import SimpleAgent, ToolRegistry, WorkingMemory`.
- The example code lives in a separate public repository: https://github.com/USAFA-AI-Center/fair_llm_demos.git.

That separation is a good thing. It means students, researchers, and outside contributors can begin using the framework through the published package and learn from the public examples without needing to browse or modify the internal source layout. From a teaching perspective, this is helpful because it keeps the first learning loop simple:

1. install the framework,
2. clone the demo repository,
3. run a demo,
4. read the code with the architecture from the earlier chapters in mind.

12.2 Recommended Environment Setup

12.2.1 A Practical Python Recommendation

The FAIR framework and its demos require Python `3.12` or higher. Some older materials in the demo repository reference `3.8+` as the minimum, but that lower bound reflects an earlier state of the project and should be disregarded. The framework is compiled and tested against Python 3.12+, and using an earlier version is likely to produce dependency conflicts or import failures.

For students, the simplest rule is this: if you are setting up a fresh environment specifically for FAIR and the demos, use Python `3.12` (or the latest available 3.x release). If you are working in an existing environment and a particular demo fails due to a version or dependency mismatch, create a new dedicated virtual environment rather than trying to repair a crowded global Python installation.

12.2.2 Framework-Only Installation

If your immediate goal is simply to verify that FAIR itself can be imported, you can install the public package directly before touching the demos. This is useful when teaching, troubleshooting, or validating a classroom environment.

```
pip install --upgrade pip
pip install fair-llm
```

Listing 12.1: Minimal FAIR package install

After installation, a quick smoke test is to open Python and verify that the public API imports cleanly:

```
from fairlib import SimpleAgent, ToolRegistry, WorkingMemory,
    HuggingFaceAdapter
print("FAIR import succeeded.")
```

Listing 12.2: Simple import smoke test

This does not yet guarantee that every demo dependency is present, but it confirms that the public package itself is available and that the `fairlib` import surface is working as expected.

12.2.3 Full Demo Environment Setup

For the full learning experience, set up the public demo repository in its own virtual environment.

```
git clone https://github.com/USAFA-AI-Center/fair_llm_demos.
    git
cd fair_llm_demos
python3.12 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python verify_setup.py
```

Listing 12.3: Recommended Linux/macOS setup for the demos

On Windows, the activation step changes slightly:

```
python -m venv .venv
.venv\Scripts\activate
pip install --upgrade pip
pip install -r requirements.txt
python verify_setup.py
```

Listing 12.4: Windows virtual environment activation

This sequence does several useful things at once. It gives you the public demo code, installs the published FAIR package along with the demo repository's baseline

requirements, and runs the included verification script so you can catch missing packages before trying to debug a specific example.

12.2.4 Environment Variables and Credentials

Several demos use local or remote model backends, and a few rely on external services. For that reason, a beginner should distinguish between *base setup* and *demo-specific credentials*.

- Hugging Face based demos require access to hosted models or local model downloads. The recommended approach is to authenticate once using the command-line tool: `huggingface-cli login`. This stores the token securely at the environment level and requires no changes to any demo script. The `HuggingFaceAdapter` accepts an optional `auth_token` parameter, but the `HF_TOKEN` environment variable is the preferred pattern and is what all current demo scripts rely on.
- The multi-agent search demos may rely on Google Custom Search Engine settings, which are documented in the repository's companion setup materials.
- Demos that rely on OpenAI, Anthropic, or other providers should be treated as environment-driven examples: configure credentials in your shell or a local `.env` file rather than hard-coding secrets into scripts.

For a classroom or lab environment, this matters because students often confuse a package installation problem with a credential problem. They are different. A script may import successfully and still fail at runtime because the model backend or search service has not been configured.

12.2.5 Optional Dependencies by Demo Family

The baseline `requirements.txt` gets you most of the way there, but certain demo families naturally need extra software or services. It is helpful to know this before you start.

- **RAG demos:** these may require packages such as `chromadb`, `sentence-transformers`, and FAISS-related dependencies.
- **Streamlit demos:** these require `streamlit` and, depending on the exact path you use, supporting packages for the specific application wrapper.
- **Local-model demos:** Hugging Face and Ollama examples may rely on local model downloads, sufficient system memory, and provider-specific runtime tools.

- **Load-balancer demos:** these require infrastructure outside the Python script itself, such as a running `vllm_load_manager` and one or more registered inference nodes.

The main practical lesson is simple: when a demo introduces a new capability, ask what that capability depends on. A RAG demo depends on document processing, embedding, and vector storage. A load-balanced demo depends on external serving infrastructure. A browser demo depends on a web application runtime. Thinking this way helps students debug systematically instead of treating every failure as a mystery.

12.3 How to Approach the Demos as a Learner

The demo repository is best understood as a progression rather than as a random pile of scripts. A new user should generally work in the following order:

1. Start with a *single-agent, single-tool* example.
2. Move to *single-agent, multi-tool* examples.
3. Study *model abstraction* and backend-swapping examples.
4. Move into *multi-agent orchestration* examples.
5. Then study *RAG*, *structured output*, and *evaluation* patterns.
6. Finally, look at *application wrappers* such as Streamlit, which show how FAIR-based logic can be placed behind a user interface.

This order matters. A learner who jumps directly into the autograders or the search-and-plot team may see too many moving pieces at once. By contrast, a learner who first understands how a single `SimpleAgent` is assembled will later be able to recognize that many larger demos are simply combinations of the same core primitives: an LLM, a planner, memory, a tool registry, and a run loop.

12.4 Foundational Single-Agent Demos

12.4.1 `demo_single_agent_calculator.py`

Focus. This is one of the best starting points in the entire repository. It shows how to build a single FAIR agent around one tool, a working memory object, a planner, and a language model adapter. If a student wants to answer the question, "What is the smallest complete FAIR agent that still feels real?" this is the demo to study first.

What it teaches. The script walks through the essential anatomy of a basic FAIR application: instantiate an LLM, register a tool in a `ToolRegistry`, wrap that registry in a `ToolExecutor`, create a `WorkingMemory`, attach a ReAct-style planner, and finally assemble the whole system into a `SimpleAgent`. The interactive loop at the end makes the architecture easy to feel in practice. The code is intentionally straightforward, which makes it ideal for classroom walkthroughs and first modifications.

Why it matters. Nearly every more advanced demo in the repository builds on this same pattern. Even when the scenario becomes more elaborate, the central logic still comes back to the same question: what model is reasoning, what tools are available, what memory is carried forward, and what planner is producing the next action?

12.4.2 `demo_advanced_calculator_calculus.py`

Focus. This demo extends the basic calculator pattern into a more capable mathematical assistant. Instead of stopping with arithmetic, it introduces symbolic calculus and shows how a single agent can coordinate multiple tools inside one reasoning loop.

What it teaches. The script combines the built-in `SafeCalculatorTool` with an `AdvancedCalculusTool`. It also customizes the planner's role definition so the agent is explicitly framed as a mathematical assistant that should reason step-by-step and choose tools carefully. For a learner, this is important because it demonstrates that FAIR is not only about wiring components together; it is also about shaping agent behavior through prompt design, role definition, and tool selection. The additional math-expression parsing logic also shows how preprocessing can improve user experience before the agent even starts reasoning.

Why it matters. This is a good second demo after the simple calculator, because it teaches how to move from "one agent, one tool" to "one agent, many capabilities" without changing the overall architectural pattern.

12.4.3 `demo_agent_config_export_load.py`

Focus. This script is about reproducibility. It shows how to save a FAIR agent's configuration to JSON and then reconstruct a working agent from that saved configuration.

What it teaches. New developers often focus only on getting an agent to run once. This demo instead teaches the next important habit: making an agent *portable*. The script builds a calculator agent, attaches prompt elements such as role definitions, format instructions, and examples, saves the resulting configuration, and then reloads it. That makes the demo especially useful for teams, coursework, and prompt-optimization workflows, because it shifts the mindset from "I built something in memory" to "I built a configuration that can be reused, shared, versioned, and evaluated."

Why it matters. As soon as a project involves more than one developer—or a student wants to preserve an agent state for later experimentation—configuration export becomes important. This demo helps learners see agent definitions as assets, not just ephemeral Python objects.

The underlying API. The save and load operations in this demo are provided by three functions in `fairlib.utils.config_manager`: `save_agent_config(agent, path)`, which serializes a `SimpleAgent`'s prompt builder, tool setup, and model settings to a JSON file; `load_agent_config(path)`, which reads that file back as a plain dictionary; and `load_agent(path, llm)`, which reconstructs a fully functional `SimpleAgent` from the saved config, given a model instance. The JSON format is shared with the `fair_prompt_optimizer` tool, so configurations produced here can flow directly into an optimization workflow and back.

12.5 Model Abstraction and Backend-Swapping Demos

12.5.1 `demo_huggingface_v4.py`

Focus. This demo steps below the "complete agent" level and looks directly at the Hugging Face adapter itself. It is a guided tour of the model abstraction layer from the perspective of a developer who wants to understand how FAIR talks to a specific backend.

What it teaches. The script demonstrates model loading, capability inspection, synchronous and asynchronous generation, streaming, and the convenience chat interface. It is especially useful for students who want to separate two ideas that are often conflated: the model adapter and the agent built on top of it. FAIR can wrap a model adapter inside a planner-driven agent, but that does not mean the adapter itself is mysterious. This demo makes that layer visible.

Why it matters. If you ever need to troubleshoot a model-specific issue, compare generation behaviors, or learn the low-level interface before introducing planning and tools, this is an excellent place to work.

12.5.2 `demo_huggingface_v5.py`

Focus. This companion demo explains what changes when the adapter is used with the Transformers v5 code path. It is less about the broader FAIR architecture and more about how FAIR remains resilient as external model libraries evolve.

What it teaches. The script discusses version detection, `TRANSFORMERS_V5`, optimized attention settings, cleaner message preparation, and version-aware async streaming. For beginners, the important lesson is not every detail of the underlying Transformers

library. The more important lesson is architectural: FAIR uses adapters precisely so the rest of your application does not have to be rewritten every time a backend library changes behavior.

Why it matters. This demo is a useful reminder that abstraction layers are not academic decoration. They are what allow an agentic framework to stay usable while models and inference libraries continue to change rapidly.

12.5.3 `demo_model_comparison.py`

Focus. This demo shows one of the clearest advantages of FAIR's model abstraction layer: the same agent pattern can be used across multiple models so their outputs can be compared side-by-side.

What it teaches. The script creates the same basic text-generation agent around different model instances and gives each one the same creative prompt. This is pedagogically useful because it isolates the variable you care about—the model—while keeping the surrounding agent structure constant. Students can therefore compare tone, quality, latency, and stylistic behavior without confusing those changes with unrelated architectural differences.

Why it matters. In real development, model selection is rarely settled once and forever. Teams compare options. They benchmark. They trade off size, cost, quality, and deployment constraints. This demo makes that workflow concrete.

12.5.4 `demo_single_agent_calculator_load_balancer.py`

Focus. This script revisits the simple calculator agent, but swaps the model backend so the agent talks to a centralized load-balanced inference service rather than a directly instantiated local adapter.

What it teaches. The architectural lesson here is subtle and important: agent logic does not need to be entangled with the serving topology underneath it. The planner, tool registry, memory, and overall run loop look familiar. What changes is the adapter that connects the agent to inference. This helps learners see why a clean model abstraction layer matters operationally. A classroom laptop, a lab GPU server, and a distributed vLLM backend can all be addressed through the same broader agent construction pattern.

Why it matters. This demo is valuable for teams moving from single-machine experimentation toward shared serving infrastructure.

12.6 Multi-Agent Coordination Demos

12.6.1 `demo_multi_agent.py`

Focus. This is the clearest introductory demo for hierarchical multi-agent work in the repository. It constructs a manager agent and a small team of workers, each with a sharply defined role.

What it teaches. The worker agents are regular FAIR agents with limited responsibilities and limited tools. The manager, by contrast, is configured with a `ManagerPlanner` so it can break the user request into subtasks and delegate work instead of solving the whole task itself. In this example, one worker handles web search while another handles mathematical analysis. The `HierarchicalAgentRunner` then orchestrates the workflow between them. For students, this is a foundational pattern: instead of imagining "one super-agent that knows everything," FAIR allows you to build teams of narrow, focused agents.

Why it matters. This demo is a bridge between basic ReAct systems and more sophisticated agentic applications. Once you understand this one, many of the more advanced committee-style examples become much easier to read.

12.6.2 `demo_multi_agent_load_balancer.py`

Focus. Whereas `demo_multi_agent.py` teaches delegation, this demo teaches scale. It launches multiple FAIR agents concurrently while they all share a common load-balanced vLLM backend.

What it teaches. Architecturally, the demo makes a distinction between *agent concurrency* and *model-serving concurrency*. The Python side uses `asyncio` to run multiple agents in parallel, while the inference side uses a central `vllm_load_manager` and one or more registered nodes. This is a very useful mental shift for intermediate developers: once you have more than one agent or more than one user, infrastructure questions matter. How is load distributed? What happens when several agents request inference at once? Where is the bottleneck?

Why it matters. It is a strong example for labs, classrooms, or internal services where many FAIR-driven tasks may need to share a common backend.

12.6.3 `demo_web_search_plot_agent.py`

Focus. This is one of the richer orchestration demos in the repository. It builds a manager-led team designed to search for information, extract usable data, and create a plot from the result.

What it teaches. The most important lesson is that agent teams should be organized around *capability boundaries*. In this demo, the system separates the search task, the

data extraction task, and the graphing task into different worker roles. The code also goes further than the simpler multi-agent example by introducing detailed capability descriptions, delegation keywords, structured worker instructions, and a more carefully shaped manager prompt. This makes it especially valuable for students who want to see how a manager can be taught to route work intelligently instead of delegating in a vague or accidental way.

Why it matters. This is a very good demo for anyone interested in building "pipeline agents" where the system moves through discovery, extraction, transformation, and presentation stages rather than answering in a single step.

12.7 Knowledge-Grounded and RAG Demos

12.7.1 `demo_rag_from_documents.py`

Focus. This is the introductory RAG demo in the repository. It shows how to ground an agent in local documents so the model can answer questions with project-specific knowledge instead of relying only on its pretrained weights.

What it teaches. The script follows the classic RAG pipeline cleanly: load a document, split it into chunks, embed the chunks, store them in a vector store, then expose retrieval as a tool the agent can use when it needs relevant context. Because the demo is modular, it also reinforces an important FAIR lesson: RAG is not one magical object. It is a composition of a loader, a splitter, an embedder, a store, a retriever, and an agent that knows how to use retrieval.

Why it matters. This is the demo to read when a student first asks, "How do I make the agent know *my* documents?" It is one of the most practical patterns in real-world agentic work.

12.7.2 `demo_faiss_rag_from_readme.py`

Focus. This script is a more advanced RAG example. It mirrors the earlier RAG demo in purpose, but changes important implementation details so students can see how different memory and retrieval choices affect the overall design.

What it teaches. Instead of using the simpler introductory path, this demo uses the `DocumentProcessor`, stores vectors in a persistent `FaissVectorStore`, and always applies re-ranking through a `CrossEncoder`. That makes it especially valuable for intermediate developers, because it shows that not all RAG stacks are equal. Retrieval quality often depends on chunking strategy, vector-store choice, persistence behavior, and post-retrieval re-ranking. This demo brings those concerns into view without abandoning the higher-level FAIR agent structure.

Why it matters. When a project moves from "toy RAG" to a more serious knowledge workflow, these are exactly the sorts of design refinements that begin to matter.

12.8 Structured Output and Evaluation Demos

12.8.1 `demo_structured_output.py`

Focus. This demo shows how FAIR can be used for reliable information extraction rather than only free-form chat. It walks through a pattern in which an LLM is asked to produce JSON that conforms to a Pydantic schema, and the output is validated before being accepted.

What it teaches. The script defines a schema, builds an extraction-oriented agent around that schema, and then applies the pattern to unstructured text. The result is a very important lesson for students: in serious applications, the model's job is often not to "sound smart," but to return structured data that another system can trust. By including validation in the loop, the demo reinforces the idea that model output should be checked and normalized before it is handed to downstream code.

Why it matters. This pattern is foundational for document extraction, workflow automation, API integration, grading pipelines, and any setting where correct structure matters as much as fluent language.

12.8.2 `demo_committee_of_agents_essay_autograder.py`

Focus. This is a sophisticated educational example that treats grading as a collaborative multi-agent workflow. Instead of one model trying to grade an essay in one pass, the system builds a committee of specialists.

What it teaches. The manager coordinates workers such as a content analyst, a clarity-and-style checker, a rubric aligner, and optionally a RAG-based fact checker grounded in course materials. The script therefore demonstrates several FAIR ideas at once: multi-agent decomposition, rubric-driven evaluation, structured final output, and optional knowledge grounding against a course corpus. For instructors, it is a concrete example of how FAIR can support repetitive, consistency-sensitive academic workflows.

Why it matters. This demo is especially useful for readers who want to see how agentic design can mirror a real human process. Instead of pretending that one model can play every role equally well, the system assigns different types of judgment to specialized workers and synthesizes the result.

12.8.3 `demo_committee_of_agents_coding_autograder.py`

Focus. This companion grading demo applies the same committee idea to code submissions. It is one of the most operationally instructive examples in the repository because it combines LLM-based review with optional code execution.

What it teaches. The system uses workers for static analysis, logic and efficiency review, rubric alignment, and optionally code execution against tests. From an agentic standpoint, this is an excellent example of mixing tool-based, verifiable signals with higher-level reasoning. From a software engineering standpoint, it also teaches an equally important negative lesson: execution of untrusted code is dangerous. The demo explicitly warns that its placeholder execution path is *not* production-safe and must be replaced with a real sandboxing mechanism in any serious deployment.

Why it matters. This is not just a grading script. It is a case study in how FAIR can support structured evaluation workflows while still respecting real engineering constraints such as testing, evidence, and security.

12.9 User-Facing Application Demos

12.9.1 `demo_streamlit_simple.py`

Focus. This script shows how FAIR-driven logic can be wrapped inside a small browser application. Instead of a terminal loop, the interaction happens through a Streamlit interface.

What it teaches. The demo uses session state, a chat-style interface, connection testing, and a local-model workflow centered around Ollama and a WolframGPT-style assistant. For students, this is useful because it marks a clear transition from "agent script" to "user-facing application." The underlying agentic ideas are still present, but now they are embedded in an interface that a non-developer could actually use.

Why it matters. Many FAIR projects will eventually need a front end, even if that front end is simple. This demo provides a gentle introduction to that step.

12.9.2 `demo_streamlit.py`

Focus. This is the richer Streamlit companion. It expands the browser experience with more formatting logic, more elaborate presentation behavior, and a stronger emphasis on rendering mathematical output cleanly.

What it teaches. In addition to the basic app pattern, the script includes logic for handling and displaying LaTeX-style mathematical content. That makes it a helpful example for developers building educational tools, tutoring systems, or technical assistants where mathematical formatting is part of the experience and not just a cosmetic afterthought.

Why it matters. It demonstrates an important maturity step in agentic application design: once you leave the terminal, presentation quality becomes part of the system design.

12.9.3 `run_streamlit_demo.sh`

Focus. Although this is not a Python demo script, it is a useful helper for new users working with the Streamlit examples.

What it teaches. The script checks for the expected environment, verifies that Streamlit is installed, installs missing dependencies when necessary, and then launches the application. For beginners, helper scripts like this reduce the amount of environment friction standing between them and a working example.

Why it matters. Good demo repositories do not only contain clever Python code; they also contain small bits of supporting infrastructure that make the examples easier to run and teach.

12.10 A Good Learning Order Through the Repository

If you are teaching FAIR to students or using the demo repository for self-instruction, the following order tends to work well:

1. `demo_single_agent_calculator.py`
2. `demo_advanced_calculator_calculus.py`
3. `demo_huggingface_v4.py` or `demo_huggingface_v5.py`
4. `demo_model_comparison.py`
5. `demo_multi_agent.py`
6. `demo_rag_from_documents.py`
7. `demo_structured_output.py`
8. `demo_web_search_plot_agent.py`
9. The autograder and Streamlit examples once the earlier ideas feel familiar.

That order is not mandatory, but it tends to align with how understanding develops. A learner usually benefits from first seeing a small, complete loop, then a slightly richer loop, then a backend abstraction, then a team of agents, then knowledge integration, then typed outputs, and finally more specialized applications.

12.11 Final Thought

The public FAIR package and the public demo repository together form a practical on-ramp into agentic development. The package gives you the reusable machinery. The demos show you how that machinery is actually assembled into working systems. For a newcomer, that combination is powerful: you can install the framework without needing the core source tree, study runnable examples, and begin experimenting almost immediately.

Used well, the demo repository is more than a collection of scripts. It is a curriculum. Each example teaches a distinct architectural lesson, and together they show how FAIR scales from a tiny one-tool agent to collaborative teams, grounded retrieval systems, structured extraction pipelines, grader workflows, and user-facing applications.

Chapter 13

The Horizon — Present Power and Future Potential

This chapter summarizes the framework's capabilities and lays out a strategic roadmap for its future evolution into a world-class, production-ready service for deploying autonomous AI. We will take stock of the powerful engine we have built and look ahead to the key engineering steps required to take this framework from a developer's toolkit to a scalable, robust, and secure server-side application capable of supporting real-world use cases.

13.1 The FAIR Framework Today - A Flexible Engine

The framework, in its current state, has successfully achieved its core goals of being modular, agnostic, consistent, and feature-rich. It is more than a collection of scripts; it is a coherent and practical engine capable of orchestrating complex and diverse agentic applications, as proven by the various demonstrations. Its present power is best understood through the key capabilities it offers out-of-the-box:

- **Advanced Agentic Reasoning (ReAct):** At its core, the framework provides a robust implementation of the Reason-Act (ReAct) loop via the SimpleAgent and ReActPlanner. This allows agents to "think" step-by-step, use tools to gather information or perform actions, and use the results of those actions to inform their next step. This foundational capability, demonstrated in scripts like demo_single_agent_calculator.py, is the engine that drives all other advanced patterns.
- **Model Agnosticism (MAL):** The Model Abstraction Layer (MAL) is a cornerstone of the framework's philosophy, ensuring developers are never locked into a single LLM provider. With concrete adapters for OpenAI, Anthropic, local Hugging Face models, Ollama, and the `LoadBalancerAdapter` for distributed vLLM inference clusters, developers can seamlessly switch between a powerful

proprietary model like GPT-4 for complex reasoning, a fast local model for simpler tasks, or a shared GPU backend for multi-user or multi-agent deployments, all without altering the core agent logic. The `demo_model_comparison.py` and load-balancer demo scripts showcase this flexibility in action.

- **Knowledge-Grounded Agents (RAG):** The framework provides a complete, end-to-end pipeline for Retrieval-Augmented Generation. As shown in demo_rag_from_documents.py, it contains all the necessary components—from document loading and chunking to embedding, vector storage (`FaissVectorStore` for production workloads, `ChromaDBVectorStore` for simpler setups), optional cross-encoder re-ranking, and retrieval (`KnowledgeBaseQueryTool`)—to ground agents in specific, external knowledge. This transforms generalist models into true subject-matter experts.

- **Sophisticated Multi-Agent Collaboration:** FAIR is designed for collaboration. The framework moves beyond single-agent systems to support hierarchical "committees of agents," as demonstrated in demo_multi_agent.py and the autograder examples. Using the HierarchicalAgentRunner and the specialized ManagerPlanner, developers can create a "manager" agent that decomposes complex tasks and delegates them to a team of specialized "worker" agents, leading to more robust and reliable solutions for multi-domain problems.

- **Reliable Structured Output:** For agents to reliably interact with other systems, they must produce predictable, structured data. The demo_structured_output.py script illustrates a powerful pattern that uses Pydantic schemas and a self-correction loop to compel the LLM to return clean, validated JSON, a feature essential for any production-grade data extraction or tool-integration task.

- **Developer-Friendly Design:** From the outset, the framework was designed for ease of use and extensibility. The central `fairlib/__init__.py` API uses a lazy-loading mechanism for faster startup times, while the interface-driven design, clear documentation, and well-commented demo scripts make the framework far easier for a development team to adopt, understand, and build upon.

13.2 The Path to Production - A Roadmap for Real-World Deployment

While the framework is a good engine, turning it into a scalable, multi-user, and secure service requires building the ***next layer of infrastructure*** around it. The key engineering challenges and the roadmap to production are clearly defined:

- **Implementing a Production API Layer:** To make the agents accessible over the internet, the framework must be wrapped in a high-performance web

server. Using a modern web framework like **FastAPI** or Flask, combined with a production-grade ASGI server like **Uvicorn** and a process manager like **Gunicorn**, would allow the system to handle concurrent API requests, where each incoming call could trigger an agent run.

- **Robust Session Management:** This is the most critical component for supporting multiple users. A production system requires:
  - **Isolation:** Each user's conversation and memory must be completely isolated from others. When a user makes a request, the system must instantiate or retrieve their specific WorkingMemory and LongTermMemory instances.
  - **Persistence:** The default in-memory conversation history is not suitable for production. User session data, including conversation history, must be persisted in a fast and reliable database. A **Redis-backed** memory implementation would be ideal for managing the short-term context of active conversations due to its speed.
- **Scalable Multi-Tenant RAG:** The in-memory ChromaDBVectorStore is excellent for demos, but a multi-user RAG application requires a persistent, server-based vector database. Integrating with a scalable vector database (such as a production deployment of Chroma, Milvus, or a cloud provider's equivalent) is necessary to handle concurrent queries and store distinct knowledge bases for many different users or tenants securely.
- **Uncompromising Security:** Security cannot be an afterthought for agentic systems. The current CodeExecutionTool, which uses a basic subprocess call, is explicitly a placeholder. A production system requires replacing this with a true sandboxing solution. This involves executing any untrusted code inside a completely isolated environment, such as a **Docker container** or a **Firecracker microVM**, to prevent it from accessing the host system.
- **A Functional Learning Module:** To enable agents to improve over time, a functional learning module is required. This involves building a feedback and data curation pipeline to capture user interactions, tool outcomes, and explicit feedback. This curated data is essential for enabling both system-level tuning (e.g., refining prompts and agent workflows) and, eventually, model-level fine-tuning to create specialized models that are highly adapted to the system's specific tasks.

13.3 Final Thought: Building the Future of AI Collaboration

The FAIR framework is built on solid architectural principles. By prioritizing modularity, clear interfaces, and extensibility from the outset, it has quickly evolved into

more than just a tool; it is a platform for sustained innovation. ***The future of artificial intelligence is not monolithic; it is collaborative***. It lies in systems of specialized agents working together, learning from their environment and *from each other*, and interacting with humans in a reliable and secure manner. The FAIR framework provides the foundation needed to build that future, offering developers the structure to build with confidence and the flexibility to innovate without limits. The path from here to a world-class service is just ahead, and it represents the next exciting chapter in the journey of agentic AI.

Chapter 14

Addendum: The Agent's Cognitive Data Pipeline

It's important to understand the precise flow of information between the components that constitute an agent's "*mind*." This process is not a single action but a multi-stage **data pipeline**, where raw information is progressively refined, enriched, and transformed into a final, actionable decision. This addendum provides a technical breakdown of the data handoffs between the `SimpleAgent`, the `Planner`, the `PromptBuilder`, the `History` (working memory), and the LLM.

14.1 The Components of the Pipeline

Each component has a distinct role in processing information:

- **Message (The Data Packet)**: The `Message` object is the fundamental unit of information in the framework. Every discrete event in a conversation—a user's query, an agent's internal reasoning, or the output from a tool—is encapsulated in a Message object. This standardization is critical for creating a consistent and reliable data flow.
- **History / WorkingMemory (The Data Store)**: An agent's WorkingMemory maintains the History, which is an ordered `List[Message]` object. It serves as the primary data store for the current session, providing a complete and chronological record of all Message packets generated during the interaction. It is the single source of truth about the immediate past.
- **PromptBuilder (The Data Transformation Engine)**: The `PromptBuilder` is a specialized component whose sole purpose is to transform raw data from the History into a precisely formatted prompt for the Language Model. It takes the agent's static configuration (its role, tool definitions, and format instructions) and combines it with the dynamic conversational History to construct the final List[Message] that the LLM will consume. This formatting step is arguably the most critical part of ensuring reliable agent behavior.

- **Planner (The Pipeline Controller)**: The `ReActPlanner` is the primary control unit and cognitive engine of the pipeline. Its ultimate responsibility is to **decide the agent's next logical step** toward achieving the user's goal. While it delegates the low-level data formatting, its core function is to analyze the current context and generate a plan. It accomplishes this through a clear sequence of operations:

  1. **Synthesize Context**: It initiates the process by sending the current History to the PromptBuilder, requesting a fully contextualized briefing for the LLM.
  2. **Generate a Plan**: It sends the resulting formatted data (the prompt) to the LLM for processing, effectively asking the LLM to generate the next Thought and Action based on the complete context.
  3. **Formulate a Command**: It receives the raw text output from the LLM and parses it into a structured, internal command—either a (Thought, Action) tuple to continue the task or a FinalAnswer to complete it. This act of parsing and structuring is what turns the LLM's raw suggestion into an executable step for the agent.

- **SimpleAgent (The Pipeline Orchestrator)**: The `SimpleAgent` is the high-level orchestrator of the entire agentic process. Its arun method is the entry point that triggers and manages the cognitive pipeline. It directs the Planner when a decision is needed, passes the Planner's commands to the ToolExecutor, and ensures the results are logged back to the History data store.

- **LLM (via MAL) (The Core Processing Unit)**: The Language Model is a powerful, general-purpose processing unit. It is completely decoupled from the framework's architecture. It receives a list of Message objects, processes the information contained within, and returns a raw string of text based on the patterns and instructions provided in the input.

14.2 The Data Pipeline: A Step-by-Step Flow

Let's trace the exact flow of data for a multi-turn task that requires memory.

Scenario: A user wants to perform a two-step calculation. The agent is equipped with a `safe_calculator` tool.

Turn 1: The Initial Calculation

- **Step 1: SimpleAgent Initiates the Pipeline** The user's query is passed to agent.arun("*What is the sum of (10/4) + 23.4 * (67/3)?*"). The SimpleAgent begins the first cognitive cycle by calling planner.aplan(history, user_input). The history is currently empty.

- **Step 2: Planner Delegates to PromptBuilder** The Planner passes the empty history and the user's user_input to its PromptBuilder.
- **Step 3: PromptBuilder Constructs the Initial Prompt** The PromptBuilder creates a List[Message] containing two items:
  1. A system message with all the agent's instructions (its role, tool definitions, format rules, and examples).
  2. The user's query as a user message: Message(role="user", content="What is the sum of (10/4) + 23.4 * (67/3)?").
- **Step 4: Planner Queries the LLM** The Planner sends this list of two messages to the LLM. The LLM analyzes the request, sees a complex math expression, and determines it should use the safe_calculator tool. It returns a raw text string like:

```
{
  "thought": "The user wants me to solve a mathematical
    expression. I should use the safe_calculator tool.",
  "action": {
    "tool_name": "safe_calculator",
    "tool_input": "(10/4) + 23.4 * (67/3)"
  }
}
```

- **Step 5: Planner Parses the LLM Output** The Planner parses this JSON string into a structured (Thought, Action) command.
- **Step 6: SimpleAgent Executes and Logs** The SimpleAgent receives the command.
  1. It sends the Action to the ToolExecutor, which runs the safe_calculator tool and gets a result, e.g., "524.3".
  2. It then logs two new messages back to WorkingMemory: the Thought (as an assistant message) and the tool's result (as a system message containing "Observation: The result of ... is 524.3").
  3. Because the Planner did not issue a FinalAnswer, the SimpleAgent loops again.

Turn 2: Reacting to the Result

- **Step 1-4: The Second Cycle** The SimpleAgent immediately starts a new cycle, calling planner.aplan(history, user_input). This time, history contains the records of the first turn, and user_input is empty. The PromptBuilder creates a

new, longer prompt containing the system message plus the full conversation history. The LLM receives this, sees that the calculation is complete and provides the answer. It returns a response indicating completion: {"thought": "I have the result of the calculation. This fully answers the user's initial request. I should provide the final answer.", "action": {"tool_name": "final_answer", "tool_input": "The sum of (10/4) + 23.4 * (67/3) is 524.3."}}.

- **Step 5: Planner Identifies FinalAnswer** The Planner parses this and recognizes the special final_answer tool, returning a FinalAnswer object to the SimpleAgent.

- **Step 6: SimpleAgent Concludes the First Task** The SimpleAgent receives the FinalAnswer object, terminates the loop, and returns the final string to the user. The History in WorkingMemory now contains the full record of this initial two-step interaction.

Turn 3: The Follow-Up Question (Demonstrating Memory)

- **Step 1: SimpleAgent Receives a New Query** The user now sends a follow-up: agent.arun("Now, add 23 to that answer"). The SimpleAgent starts a new pipeline run.

- **Step 2 & 3: PromptBuilder Uses the Memory (History)**

The Planner calls the PromptBuilder, passing it the **full, existing History** and the new user_input. The PromptBuilder constructs a prompt that contains the entire previous conversation, including the final answer of "524.3", followed by the new user request.

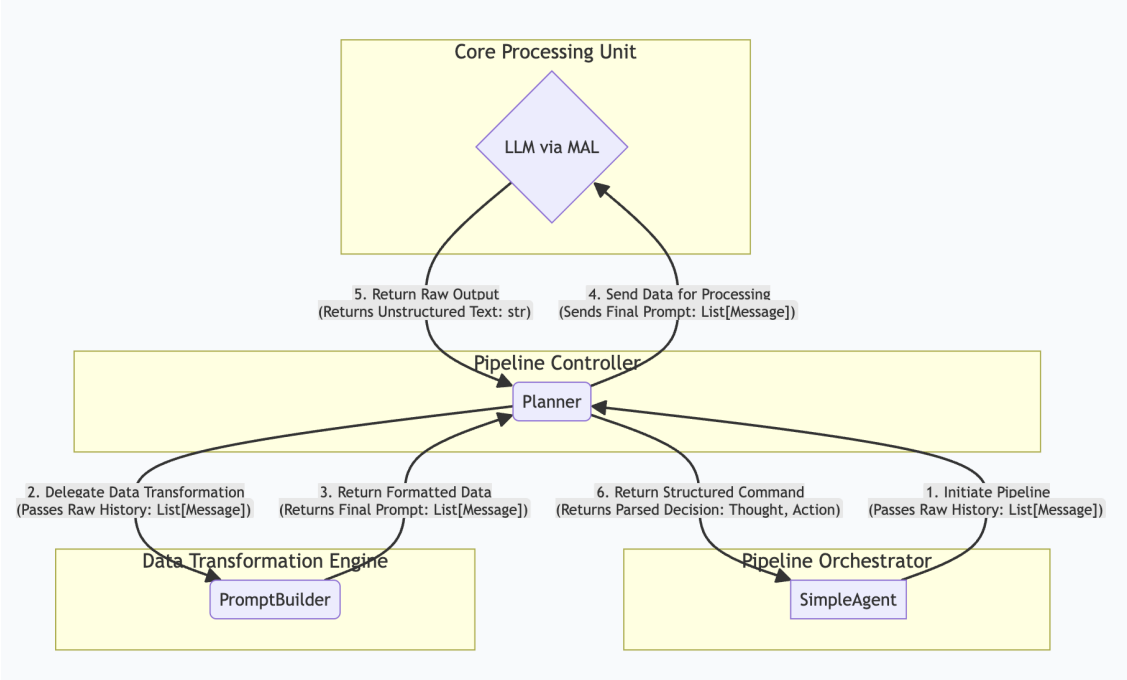
- **Step 4: Planner Queries the LLM with Full Context** The LLM receives this complete context. It sees the new request "add 23 to that answer" and, by looking at the preceding messages in the prompt, it knows that "that answer" refers to "524.3". It generates a new command:

```
{
  "thought": "The user wants to add 23 to the previous
    result of 524.3. I need to use the calculator again.",
  "action": {
    "tool_name": "safe_calculator",
    "tool_input": "524.3 + 23"
  }
}
```

- **Step 5 & 6: The Pipeline Completes** The process repeats as before. The Planner parses the command, the SimpleAgent executes the tool, gets a result ("547.3"), and the LLM generates a final answer on the subsequent loop. The crucial point is that the History provided the necessary context for the LLM to understand the user's ambiguous request.

14.3 Visualizing the Data Pipeline

This diagram illustrates the data transformations and handoffs at each stage of the cognitive process:



This multi-stage pipeline emphasizes separation of concerns, and it is a core architectural design of the FAIR framework. It helps to ensure that data is handled consistently, that responsibilities are cleanly divided, and that the system as a whole is more reliable, debuggable, and extensible.

14.4 Practical Implementation Notes

A few implementation details are worth keeping in mind as you work through the framework and companion demos:

- The main prompting implementation lives under `modules/prompting/`. In practice, `PromptBuilder` and the prompt-item classes there are the center of prompt assembly work.

- `DocumentProcessor` is the main ingestion utility used for document extraction and chunk preparation in the RAG workflow.
- FAIR supports both ChromaDB and FAISS-style retrieval paths, and the FAISS path can be paired with `CrossEncoderRerankingRetriever` when you want a second-stage reranking pass.
- The public import surface exposed through `fairlib` is the intended starting point for most users. Direct module imports are still useful when you want tighter control or need a more specialized component.
- In the current `SimpleAgent` implementation, many demos assign the role description after construction. That is why you may see worker agents in manager-worker examples receive their role text explicitly in the demo code.
- The companion demos may include convenience helpers or integration patterns that sit alongside the framework core rather than inside the smallest core package footprint. Read them as examples of intended usage patterns, not just as a one-to-one map of where every helper must live.
- `CodeExecutionTool` should still be treated as a demo or lab component until it is run inside a proper sandbox.